الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

College of Computing and Informatics

Introduction to Database

xxxxx

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details: **CRN: 50245**

Name: Sultan Aljohani **ID: 240035689**
Name: Amjad Najmi **ID: 240020509**
Name: Abdulaziz Alharbi **ID: 250021379**

Instructions:

- You must submit two separate copies **(one Word file and one PDF file)** using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, convert text into image or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You MUST show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
  - Database Design. **(6 marks)**
  - Database Implementation. **(6 marks)**
  - SQL Operations. **(6 marks)**
  - Project Reflection. **(2 marks)**
- Each leader in the group must submit one report about their chosen Project via the Blackboard **(Email submission will not be accepted and will be awarded ZERO marks)** containing the following:
  A. Task 1 : database design
  B. Task 2: database implementation
  C. All requested queries/results
  D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
  E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You MUST show all your work, and text **must not** be converted into an image unless specified otherwise by the question.

- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

Project Description

Learning Outcome(s):

CLO1, CLO2, CLO3, CLO4, CLO5

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Task 1 – Database Design (6 Marks)

- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.

Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
4. What would you do differently if you were to redo this project with more time or resources?

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
    - Live report document (with edit history): [link]
    - GitHub/GitLab repository (with commit history): [link]
    - Google Colab notebook (if applicable): [link]

Table of Contents

Table of Contents

Project Overview

Background

A private outpatient clinic that relies on manual records can experience fragmented patient information, duplicated data, appointment conflicts, and difficulty connecting clinical services with payment activity. The Smart Clinic Database System addresses these issues through one structured relational database that organizes the clinic's people, specialties, appointments, consultations, prescriptions, medication inventory, and payments.

Project Objective and Scope

The project designs, implements, and verifies a complete MySQL 8.0 database using enhanced entity-relationship modeling, relational mapping, integrity constraints, linked fictional Saudi-context records, required SQL operations, a reusable financial view, and an inventory-control trigger. Its scope covers database creation, sample-data population, code-and-result evidence, controlled transaction tests, documentation, and organized repository artifacts.

Team Task Distribution

Student	ID	Primary Assignment	Primary Evidence
Sultan Aljohani	**240035689**	Group Leader and Repository Manager; Task 1	Project coordination, repository organization, EER design, relational mapping, and integration review.
Amjad Najmi	**240020509**	Task 2: Database Implementation	Database creation, ten table definitions, constraints, fictional data, and populated-table verification.
Abdulaziz Alharbi	**250021379**	Task 3: SQL Operations	SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, TRIGGER, testing, and evidence.
All members	—	Task 4 and final review	Reflection, cross-review, testing, final documentation, and submission checking.

1. Task 1 - Database Design

The conceptual design contains ten relations and directly maps the clinic's main operational rules into a normalized relational structure.

1.1 Entity Catalogue

Entity	Primary Key	Purpose
CLINIC_PERSON	person_id	Supertype for identity, contact, demographic, and city information shared by patients and doctors.
PATIENT	patient_id	Stores medical record, blood group, insurance, emergency contact, and registration data.
SPECIALTY	specialty_id	Controls specialty names, clinic zones, and standard appointment-slot durations.
DOCTOR	doctor_id	Stores specialty assignment, professional license, office, fee, hire date, and availability.
APPOINTMENT	appointment_id	Schedules one patient with one doctor and records status, source, reason, and planned duration.
CONSULTATION	consultation_id	Records the completed clinical assessment, diagnosis, service fee, and follow-up date.
MEDICATION	medication_id	Stores medication identity, dosage form, price, stock, reorder threshold, and prescription requirement.
PRESCRIPTION	prescription_id	Stores a prescription header issued during a consultation.
PRESCRIPTION_ITEM	prescription_id + medication_id	Associative relation storing dose, frequency, course, route, instructions, and dispensed units.
PAYMENT	payment_id	Records appointment payments, method, status, payer type, time, amount, and receipt number.

1.2 EER Specialization/Generalization

CLINIC_PERSON is the supertype for shared identity and contact attributes. PATIENT and DOCTOR are modeled as total and disjoint subtypes at the application level: every clinic person in the project belongs to one subtype, and the same person is not simultaneously stored as both. Each subtype reuses person_id as its primary key and foreign key, reducing duplication while retaining role-specific attributes.

1.3 Relationships and Cardinalities

Relationship	Cardinality	Business Rule
CLINIC_PERSON - PATIENT	1 : 0..1	A clinic person may appear once as a patient subtype.
CLINIC_PERSON - DOCTOR	1 : 0..1	A clinic person may appear once as a doctor subtype.
SPECIALTY - DOCTOR	1 : 0..N	A specialty may classify many doctors; every doctor has one specialty.
PATIENT - APPOINTMENT	1 : 0..N	A patient may have many appointments; each appointment has one patient.
DOCTOR - APPOINTMENT	1 : 0..N	A doctor may have many appointments; each appointment has one doctor.
APPOINTMENT - CONSULTATION	1 : 0..1	An appointment may produce at most one consultation.
APPOINTMENT - PAYMENT	1 : 0..N	An appointment may have several payment transactions.
CONSULTATION - PRESCRIPTION	1 : 0..N	A consultation may issue multiple prescriptions.
PRESCRIPTION - PRESCRIPTION_ITEM	1 : 0..N	A prescription may contain multiple medication items.
MEDICATION - PRESCRIPTION_ITEM	1 : 0..N	A medication may appear in multiple prescription items.

1.4 Relational Schema

Relation	Relational Schema Notation
CLINIC_PERSON	CLINIC_PERSON(person_id PK, national_identifier UK, first_name, last_name, birth_date, sex, mobile_number UK, email_address UK, city)
PATIENT	PATIENT(patient_id PK/FK → CLINIC_PERSON.person_id, medical_record_no UK, blood_group, insurance_company, emergency_contact_name, emergency_mobile, registered_on)
SPECIALTY	SPECIALTY(specialty_id PK, specialty_name UK, clinic_zone, standard_slot_minutes)
DOCTOR	DOCTOR(doctor_id PK/FK → CLINIC_PERSON.person_id, specialty_id FK → SPECIALTY.specialty_id, professional_license UK, office_code UK, hired_on, consultation_fee, availability_status)
APPOINTMENT	APPOINTMENT(appointment_id PK, patient_id FK → PATIENT.patient_id, doctor_id FK → DOCTOR.doctor_id, starts_at, planned_minutes, appointment_status, booking_source, visit_reason, booked_at; UK doctor/time and patient/time)
CONSULTATION	CONSULTATION(consultation_id PK, appointment_id FK/UK → APPOINTMENT.appointment_id, consulted_at, diagnosis, assessment_notes, service_fee, follow_up_date)
MEDICATION	MEDICATION(medication_id PK, generic_name, brand_name, strength, dosage_form, unit_price, stock_on_hand, reorder_level, requires_prescription; UK brand/strength/form)
PRESCRIPTION	PRESCRIPTION(prescription_id PK, consultation_id FK → CONSULTATION.consultation_id, issued_at, general_instructions)
PRESCRIPTION_ITEM	PRESCRIPTION_ITEM(prescription_id PK/FK → PRESCRIPTION.prescription_id, medication_id PK/FK → MEDICATION.medication_id, dose_description, frequency_per_day, course_days, units_dispensed, administration_route, item_instructions)
PAYMENT	PAYMENT(payment_id PK, appointment_id FK → APPOINTMENT.appointment_id, amount, recorded_at, payment_method, payment_status, payer_type, receipt_number UK)

1.5 Complete ER/EER Diagram

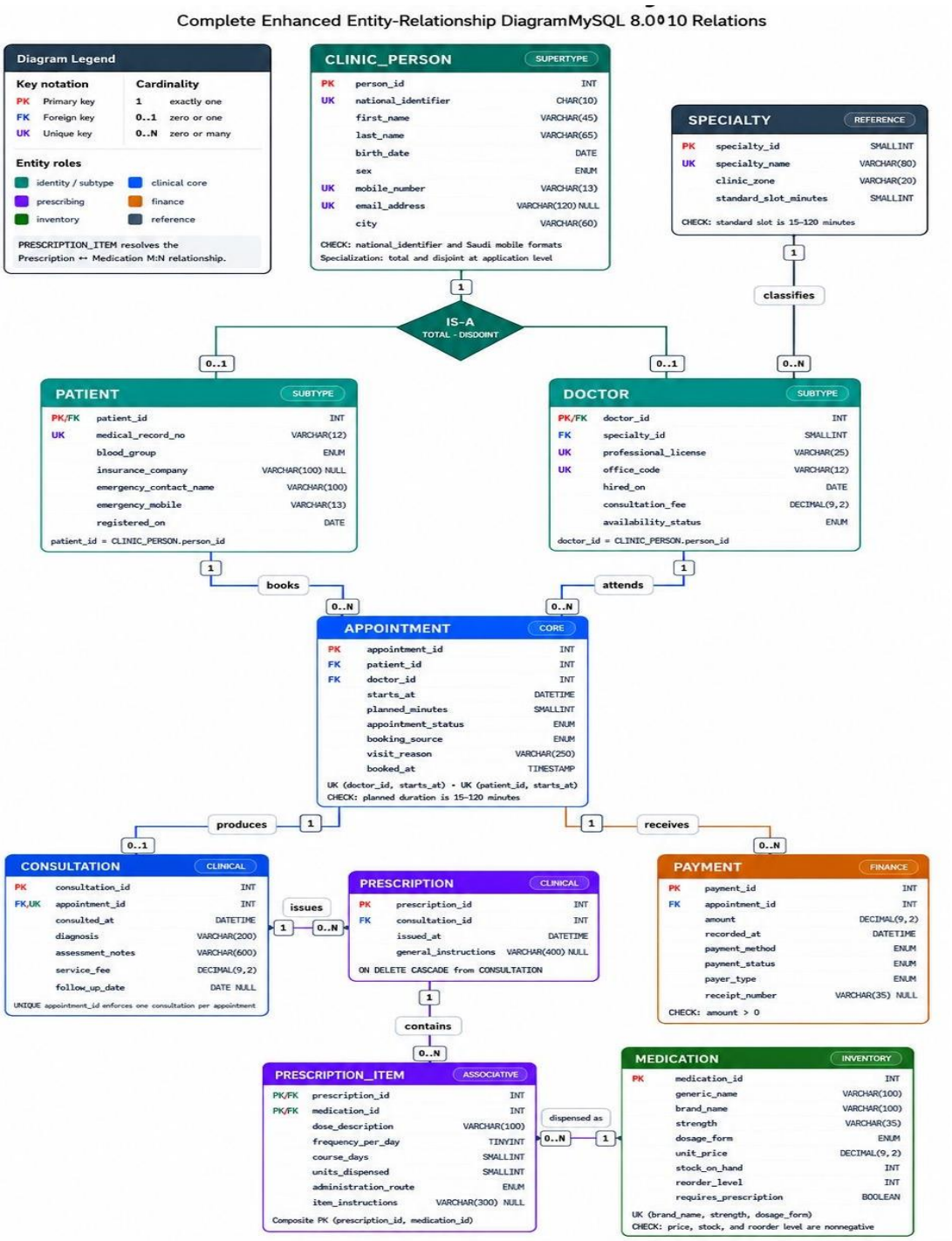

Figure 1. Complete Smart Clinic ER/EER diagram (diagrams/er_diagram.png).

1.6 Design Assumptions

- All personal, clinical, prescription, inventory, and payment records are fictional and used only for academic testing.
- The total and disjoint CLINIC_PERSON specialization is maintained by the application and verified sample data; each clinic person is assigned exactly one role.
- Every appointment references exactly one registered patient and one active or recorded doctor.
- An appointment can exist before a clinical outcome is recorded, but it can have no more than one consultation.
- A consultation can generate several prescription headers, and each prescription can contain several medication items.
- The composite primary key in PRESCRIPTION_ITEM prevents the same medication from being repeated in one prescription.
- Multiple payments are permitted for one appointment to support patient, insurance, and split-payment scenarios.
- Clinical and financial history uses restrictive parent deletion, while dependent prescription rows cascade when their clinical parent is removed.
- Doctor-time and patient-time unique constraints prevent duplicate bookings at the same start time.

2 Task 2 - Database Implementation

The conceptual design was translated into one repeatable MySQL script that creates the database, applies integrity constraints, inserts linked data, and provides verification statements for every table.

2.1 Implementation Environment

Item	Implementation
Database management system	MySQL 8.0 or newer
Development interface	MySQL Workbench
Database name	smart_clinic_system
Storage engine	InnoDB
Character set and collation	utf8mb4 / utf8mb4_0900_ai_ci
Implementation file	database/smart_clinic_database.sql

2.2 Data Types and Integrity Constraints

Constraint or Feature	Purpose in the Database
PRIMARY KEY	Uniquely identifies every entity and transaction; PRESCRIPTION_ITEM uses a composite primary key.
FOREIGN KEY	Preserves valid references between people, specialties, appointments, consultations, prescriptions, medications, and payments.
NOT NULL	Requires essential identity, clinical, scheduling, stock, and financial values.
UNIQUE	Prevents duplicate identifiers, contact values, licenses, offices, medical records, receipts, products, and appointment slots.
CHECK	Validates national/mobile formats, positive fees and amounts, valid stock, durations, frequencies, and dispensed units.
ENUM and BOOLEAN	Restricts controlled values such as status, source, method, payer, route, dosage form, sex, and availability.
CASCADE / RESTRICT	Balances dependent cleanup with protection of clinical and financial history.

2.3 Sample Data Population

Table	Rows	Population Summary
CLINIC_PERSON	10	Five patients and five doctors share the person supertype.
PATIENT	5	Five fictional Saudi-context patient profiles.
SPECIALTY	5	Family medicine, cardiology, dermatology, pediatrics, and dentistry.
DOCTOR	5	One doctor assigned to each specialty.
APPOINTMENT	7	Five completed, one booked, and one cancelled appointment.
CONSULTATION	5	One consultation for each completed appointment.
MEDICATION	7	Prescription and non-prescription products with stock controls.
PRESCRIPTION	5	One prescription for each recorded consultation in the sample.
PRESCRIPTION_ITEM	7	Dosage and course details linking prescriptions to medications.
PAYMENT	7	Paid and pending transactions from patients and insurers.

2.4 Execution Sequence

1) Drop and recreate smart_clinic_system using utf8mb4.
2) Create parent relations first: CLINIC_PERSON and SPECIALTY.
3) Create PATIENT and DOCTOR, followed by APPOINTMENT and CONSULTATION.
4) Create MEDICATION, PRESCRIPTION, PRESCRIPTION_ITEM, and PAYMENT in dependency order.
5) Insert fictional sample data from parent relations to dependent relations inside a transaction.
6) Commit the baseline dataset and run SHOW TABLES and ordered SELECT statements for verification.
7) Create and test the reporting view and inventory trigger after the core data is available.

2.5 MySQL Workbench Evidence

The following MySQL Workbench screenshots verify the schema objects and every populated table. The original captures are retained without removing the lower taskbar area so the execution date and time remain visible.

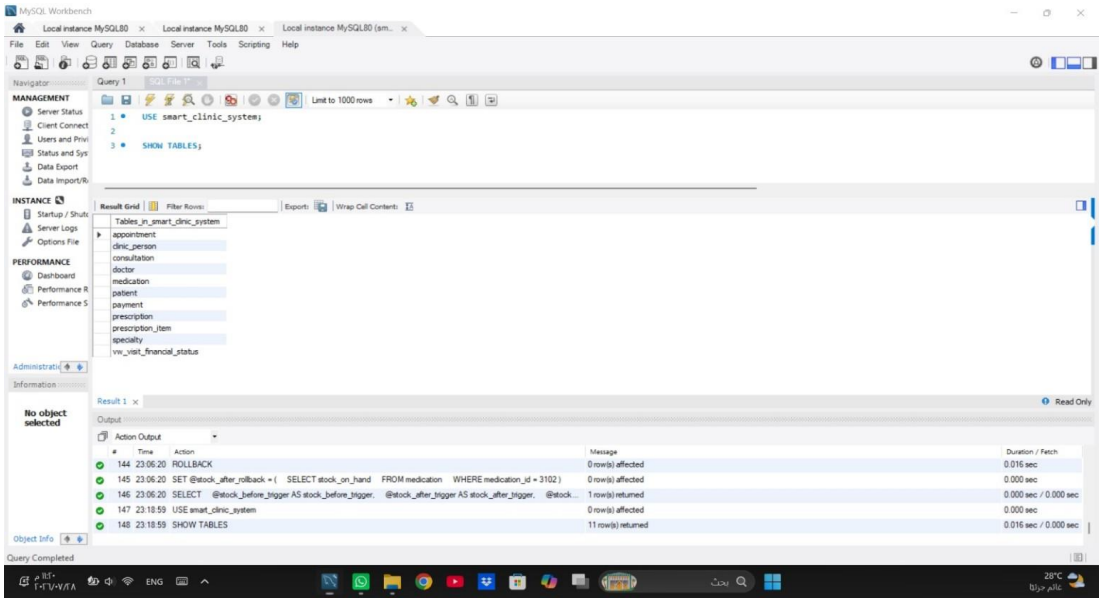

Figure 2.1. Database inventory showing 10 tables and vw_visit_financial_status.

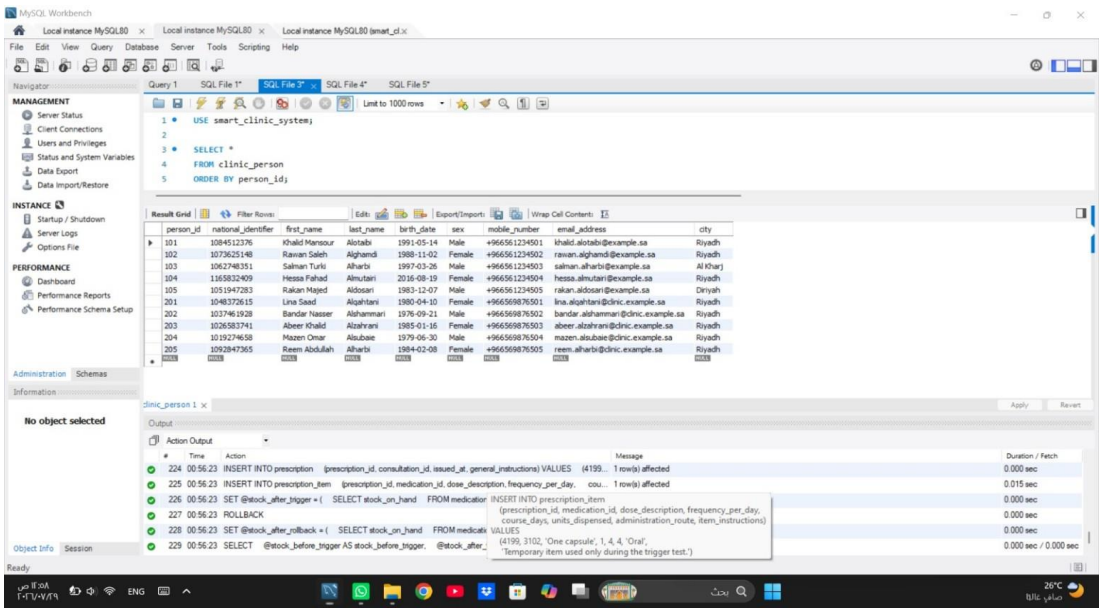

Figure 2.2. CLINIC_PERSON table containing 10 records.

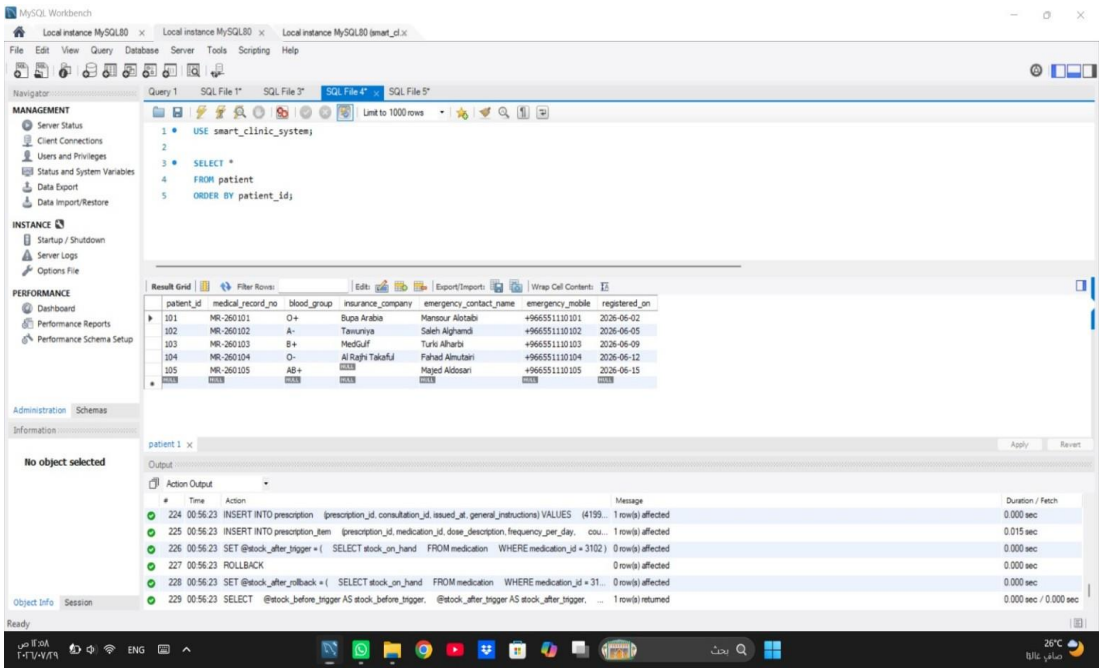

Figure 2.3. PATIENT table containing 5 records.

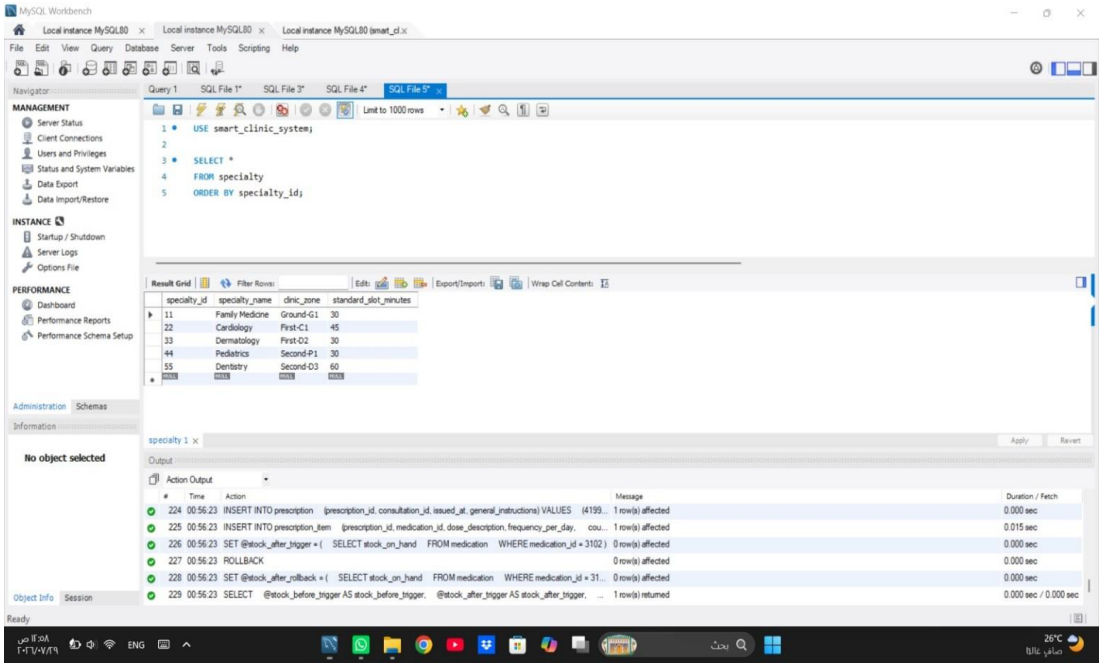

Figure 2.4. SPECIALTY table containing 5 records.

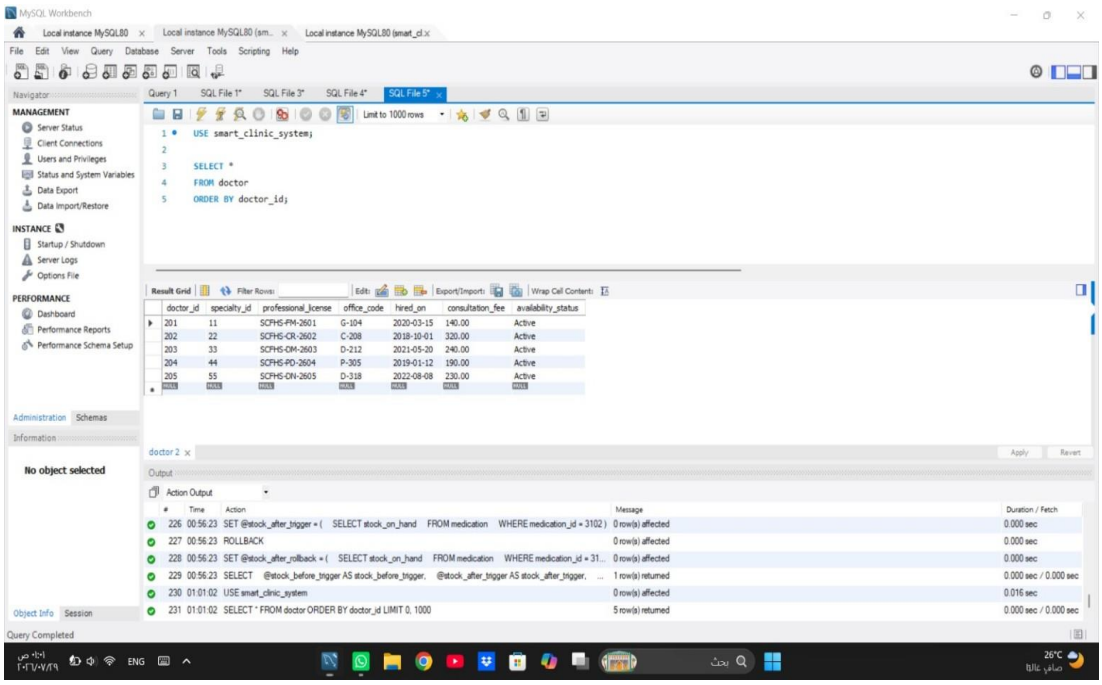

Figure 2.5. DOCTOR table containing 5 records.

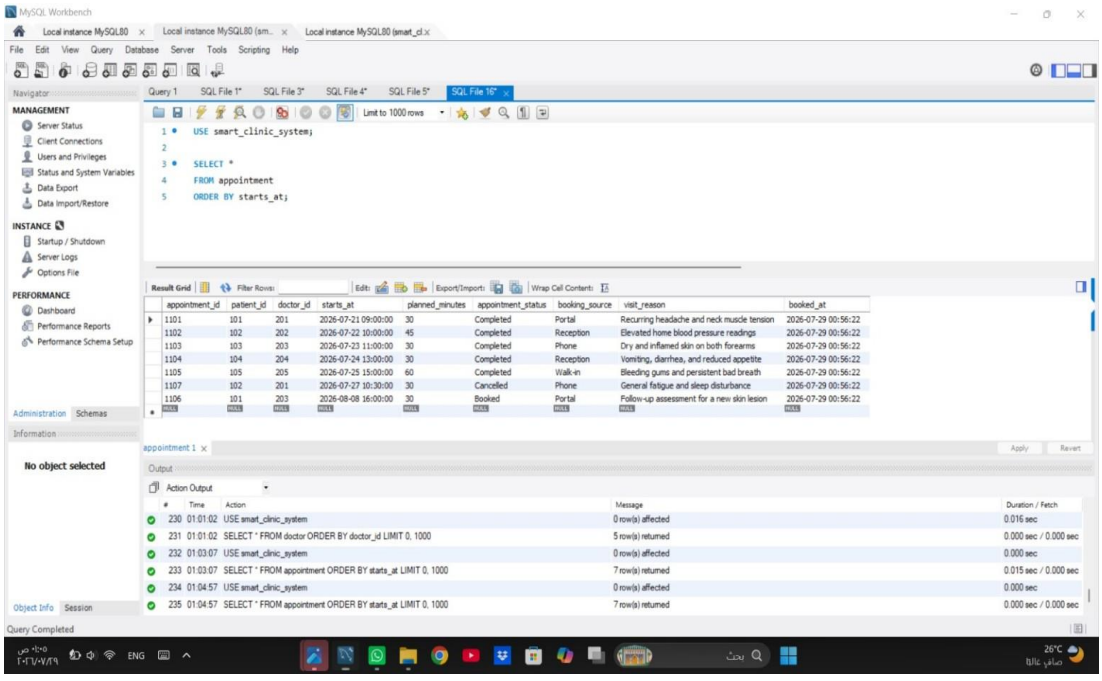

Figure 2.6. APPOINTMENT table containing 7 records.

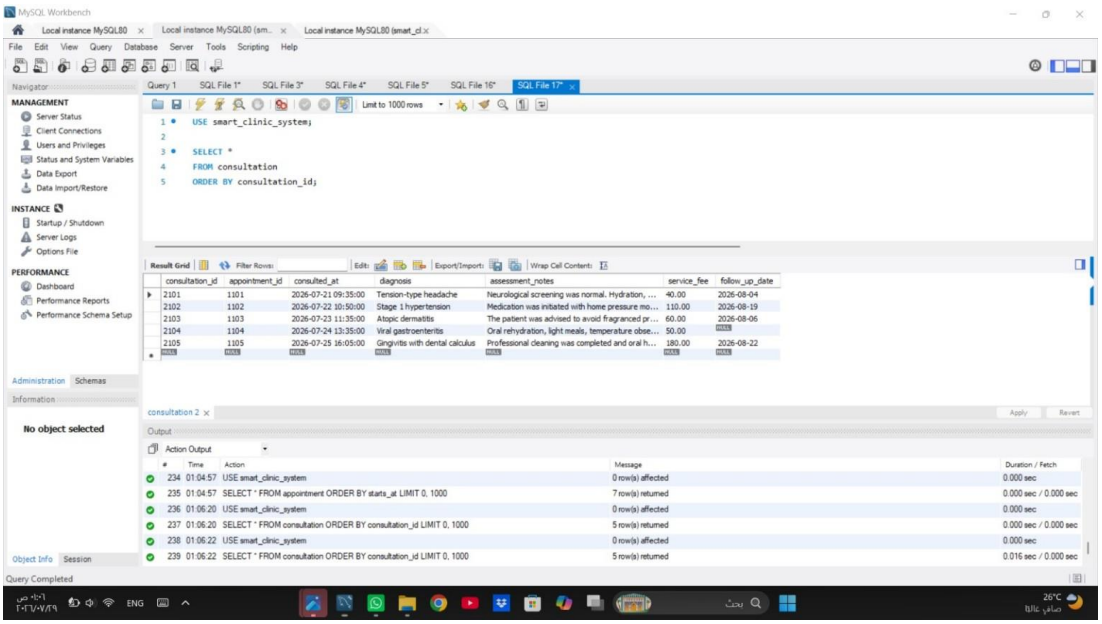

Figure 2.7. CONSULTATION table containing 5 records.

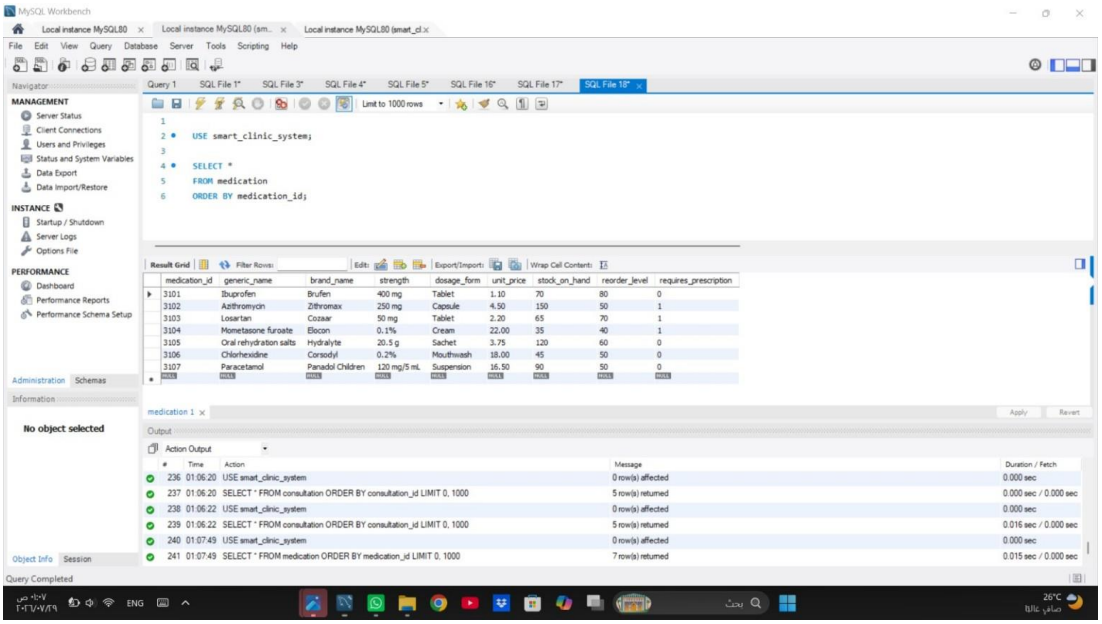

Figure 2.8. MEDICATION table containing 7 records.

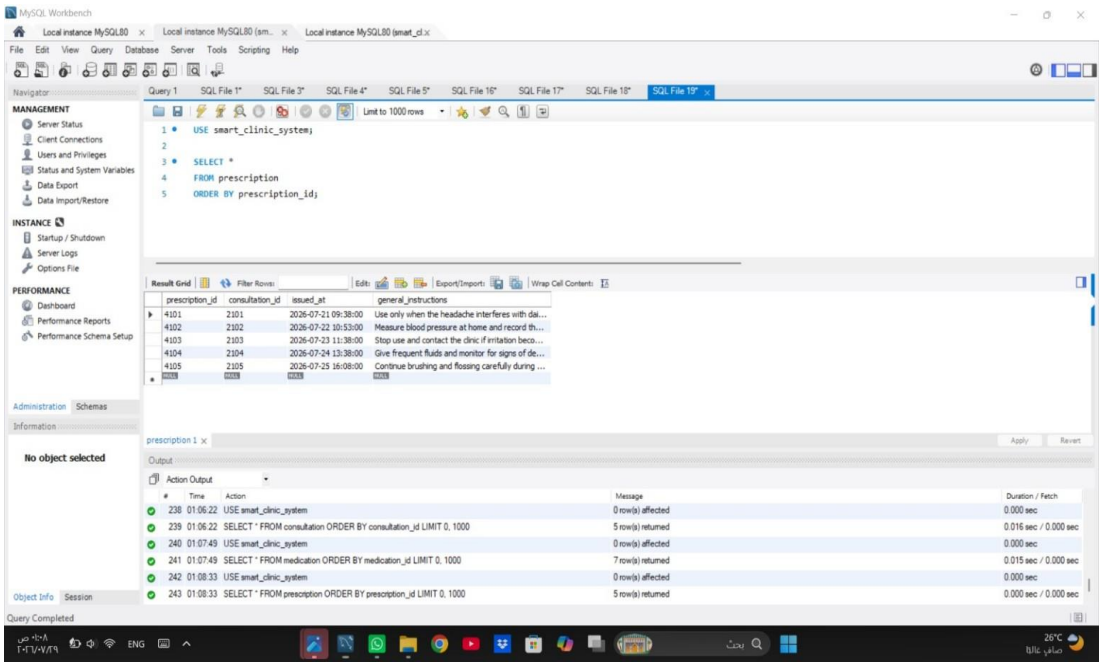

Figure 2.9. PRESCRIPTION table containing 5 records.

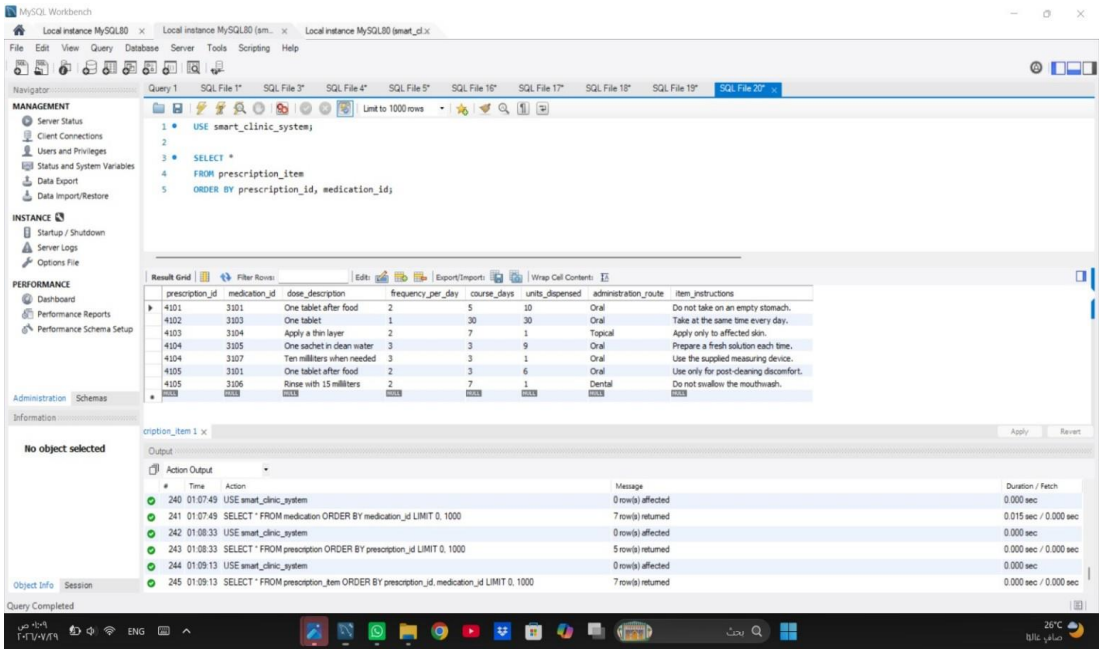

Figure 2.10. PRESCRIPTION_ITEM table containing 7 records.

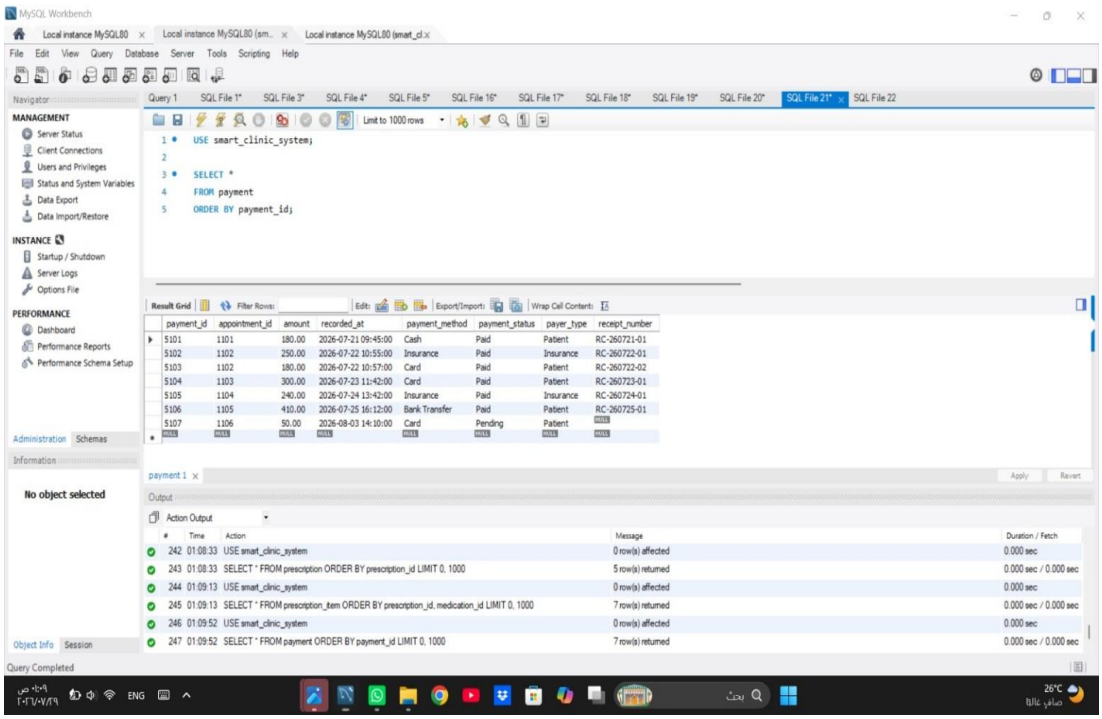

Figure 2.11. PAYMENT table containing 7 records.

3 Task 3 - SQL Operations

Each assessed operation was executed in MySQL Workbench and paired with readable code-and-result evidence. UPDATE, DELETE, and TRIGGER tests use transactions and ROLLBACK so the verified baseline data remains unchanged.

3.1 Verified Result Summary

Operation	Verified MySQL Result
SELECT - completed visits	5 completed appointments.
SELECT - stock monitoring	4 medications at or below their reorder levels.
JOIN - appointment details	7 appointment rows with patient, doctor, specialty, and status.
JOIN - patient prescriptions	7 rows linking patients, diagnoses, medication, dosage, and route.
Nested query	Patients 102 and 105 are above the average total paid per paying patient.
GROUP BY	5 doctor summary rows.
UPDATE	No insurance → Arabian Shield Cooperative → No insurance.
DELETE	Rows before/deleted/after deletion/after rollback = 1 / 1 / 0 / 1.
VIEW	7 visit-level financial rows.
TRIGGER	Medication stock = 150 → 146 → 150 after rollback.

3.2 SELECT Statements

The first SELECT produces the completed-visit register in chronological order. The second identifies products that have reached or fallen below the defined reorder threshold.

```
USE smart_clinic_system;

SELECT
    appointment_id,
    starts_at,
    planned_minutes,
    booking_source,
    visit_reason
FROM appointment
WHERE appointment_status = 'Completed'
```

```
ORDER BY starts_at;

USE smart_clinic_system;

SELECT
    medication_id,
    brand_name,
    strength,
    stock_on_hand,
    reorder_level,
    (reorder_level - stock_on_hand) AS units_below_target
FROM medication
WHERE stock_on_hand <= reorder_level
ORDER BY stock_on_hand, brand_name;
```

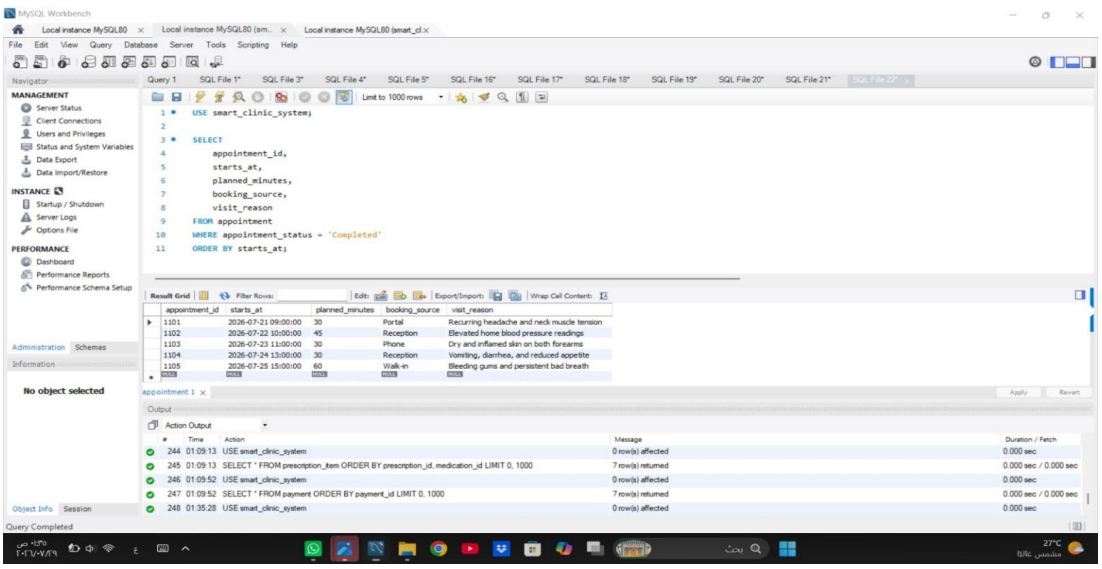

Figure 3.1. Completed appointments SELECT result: 5 rows.

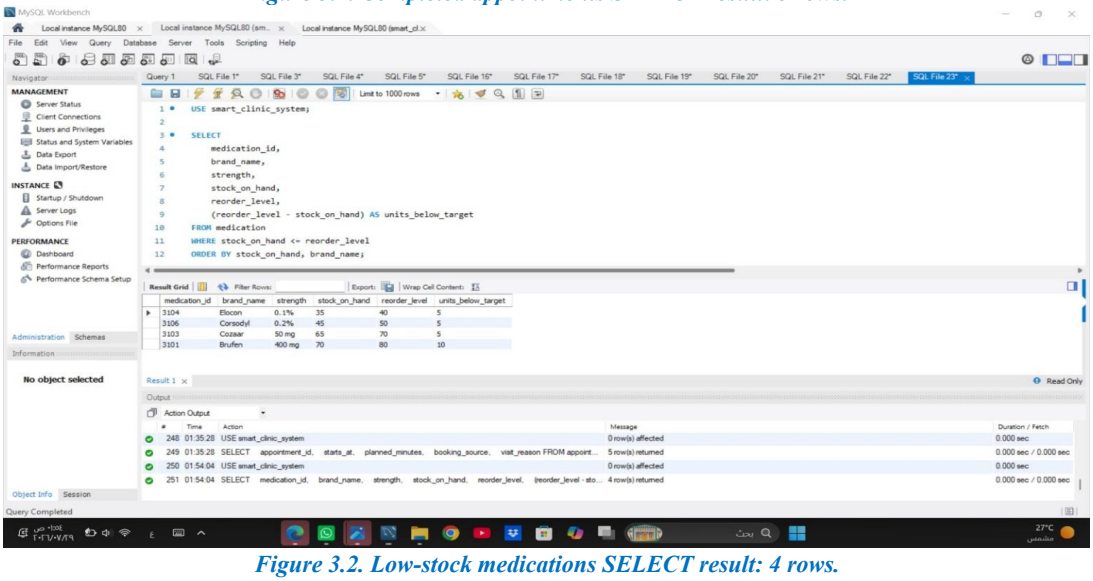

Figure 3.2. Low-stock medications SELECT result: 4 rows.

The results support both clinical visit tracking and inventory replenishment decisions.

3.3 JOIN Query - Appointment Details

This query combines six relations to present each appointment with readable patient and doctor names, the doctor's specialty, time, and status.

```sql
USE smart_clinic_system;

SELECT
    a.appointment_id,
    a.starts_at,
    CONCAT(pp.first_name, ' ', pp.last_name) AS patient_name,
    CONCAT(dp.first_name, ' ', dp.last_name) AS doctor_name,
    s.specialty_name,
    a.appointment_status
FROM appointment AS a
JOIN patient AS p
    ON p.patient_id = a.patient_id
JOIN clinic_person AS pp
    ON pp.person_id = p.patient_id
JOIN doctor AS d
    ON d.doctor_id = a.doctor_id
JOIN clinic_person AS dp
    ON dp.person_id = d.doctor_id
JOIN specialty AS s
    ON s.specialty_id = d.specialty_id
ORDER BY a.starts_at;
```

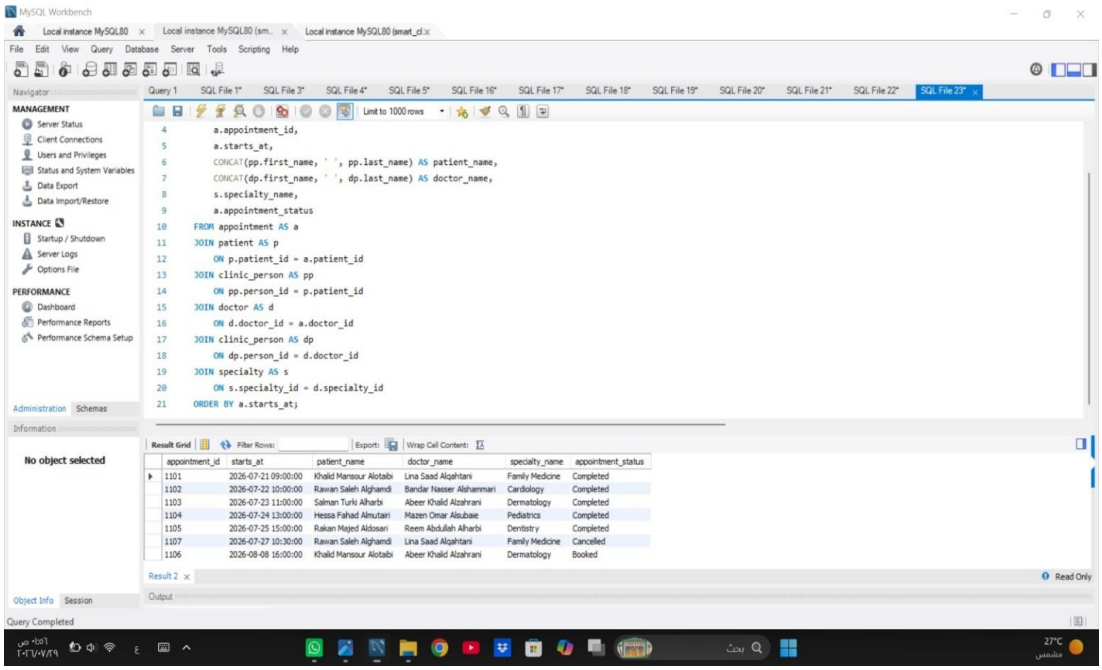

Figure 3.3. Appointment details JOIN result: 7 rows.

All seven appointments are returned without duplicating appointment identifiers.

3.4 JOIN Query - Patient Prescriptions

This query follows the clinical path from patient to appointment, consultation, prescription, prescription item, and medication so each issued treatment can be interpreted in context.

```
USE smart_clinic_system;

SELECT
    pat.medical_record_no,
    CONCAT(cp.first_name, ' ', cp.last_name) AS patient_name,
    c.diagnosis,
    m.brand_name AS medication,
    m.strength,
    pi.dose_description,
    pi.frequency_per_day,
    pi.course_days,
    pi.administration_route
FROM patient AS pat
JOIN clinic_person AS cp
    ON cp.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN consultation AS c
    ON c.appointment_id = a.appointment_id
JOIN prescription AS pr
    ON pr.consultation_id = c.consultation_id
JOIN prescription_item AS pi
    ON pi.prescription_id = pr.prescription_id
JOIN medication AS m
    ON m.medication_id = pi.medication_id
ORDER BY patient_name, medication;
```

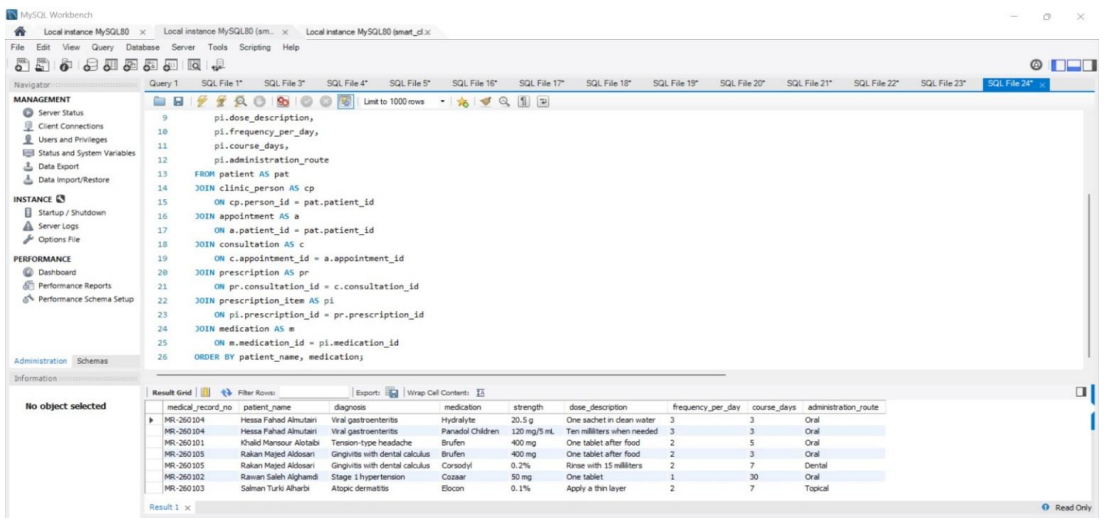

Figure 3.4. Patient prescriptions JOIN result: 7 rows.

The result contains seven prescribed medication items with their diagnosis, strength, dose, frequency, duration, and route.

3.5 Nested Query

The nested query first calculates each paying patient's completed payment total, then compares the outer patient total with the average of those grouped totals rather than with the average individual payment.

```
USE smart_clinic_system;
SELECT
    pat.patient_id,
    pat.medical_record_no,
    CONCAT(cp.first_name, ' ', cp.last_name) AS patient_name,
    SUM(pay.amount) AS patient_paid_total
FROM patient AS pat
JOIN clinic_person AS cp
    ON cp.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
WHERE pay.payment_status = 'Paid'
GROUP BY pat.patient_id, pat.medical_record_no,
         cp.first_name, cp.last_name
HAVING SUM(pay.amount) > (
    SELECT AVG(patient_total)
    FROM (
        SELECT
            a2.patient_id,
            SUM(pay2.amount) AS patient_total
        FROM appointment AS a2
        JOIN payment AS pay2
            ON pay2.appointment_id = a2.appointment_id
        WHERE pay2.payment_status = 'Paid'
        GROUP BY a2.patient_id
    ) AS paid_by_patient
)
ORDER BY patient_paid_total DESC;
```

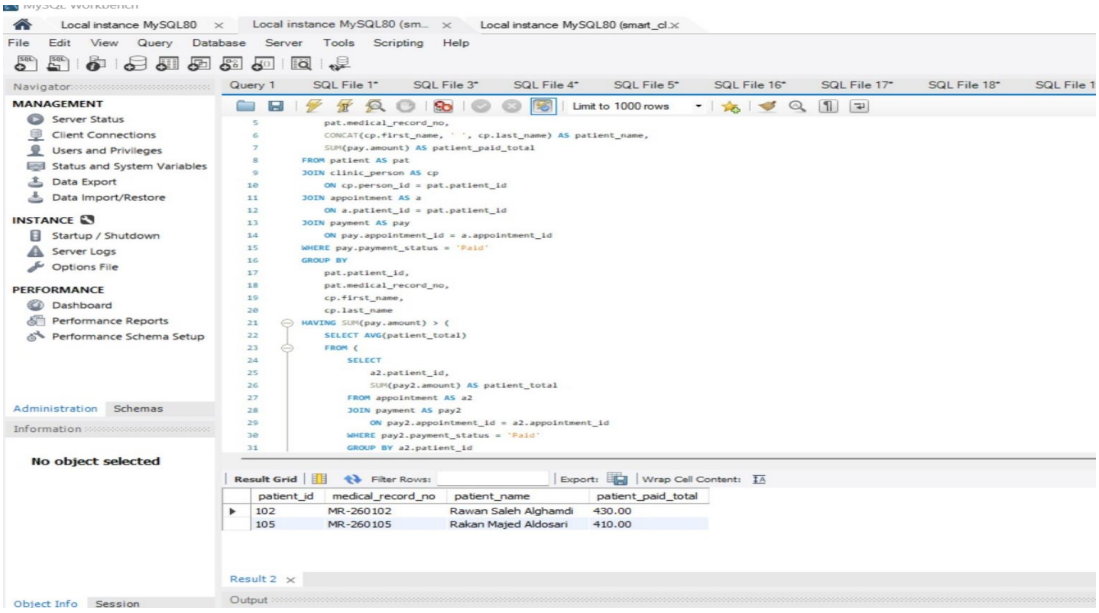

Figure 3.5. Nested query result: patient IDs 102 and 105.

3.6 Aggregate Functions with GROUP BY

The aggregate query summarizes each doctor's bookings, completed visits, average completed-visit duration, visit charges, and collected amount. Payments are aggregated by appointment before the main join to prevent duplicated totals.

```sql
USE smart_clinic_system;

SELECT
    d.doctor_id,
    CONCAT(cp.first_name, ' ', cp.last_name) AS doctor_name,
    s.specialty_name,
    COUNT(a.appointment_id) AS total_bookings,
    SUM(CASE WHEN a.appointment_status = 'Completed'
             THEN 1 ELSE 0 END) AS completed_visits,
    ROUND(AVG(CASE WHEN a.appointment_status = 'Completed'
                   THEN a.planned_minutes END), 2)
        AS average_completed_minutes,
    COALESCE(SUM(CASE WHEN a.appointment_status = 'Completed'
                      THEN d.consultation_fee + COALESCE(c.service_fee, 0)
                      ELSE 0 END), 0.00)
        AS completed_visit_charges,
    COALESCE(SUM(pt.net_paid), 0.00) AS net_collected
FROM doctor AS d
JOIN clinic_person AS cp
    ON cp.person_id = d.doctor_id
JOIN specialty AS s
    ON s.specialty_id = d.specialty_id
LEFT JOIN appointment AS a
    ON a.doctor_id = d.doctor_id
LEFT JOIN consultation AS c
    ON c.appointment_id = a.appointment_id
LEFT JOIN (
    SELECT
        appointment_id,
        SUM(CASE
                WHEN payment_status = 'Paid' THEN amount
                WHEN payment_status = 'Refunded' THEN -amount
                ELSE 0
            END) AS net_paid
    FROM payment
    GROUP BY appointment_id
) AS pt
    ON pt.appointment_id = a.appointment_id
GROUP BY d.doctor_id, cp.first_name, cp.last_name, s.specialty_name
ORDER BY completed_visits DESC, doctor_name;
```

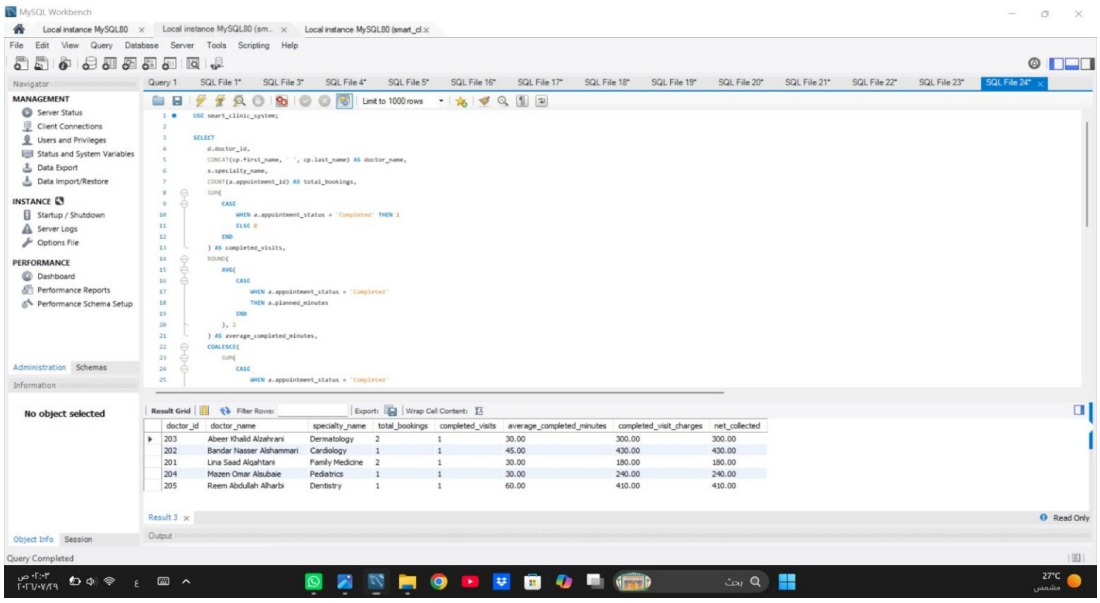

Figure 3.6A. Doctor GROUP BY evidence - Part A.

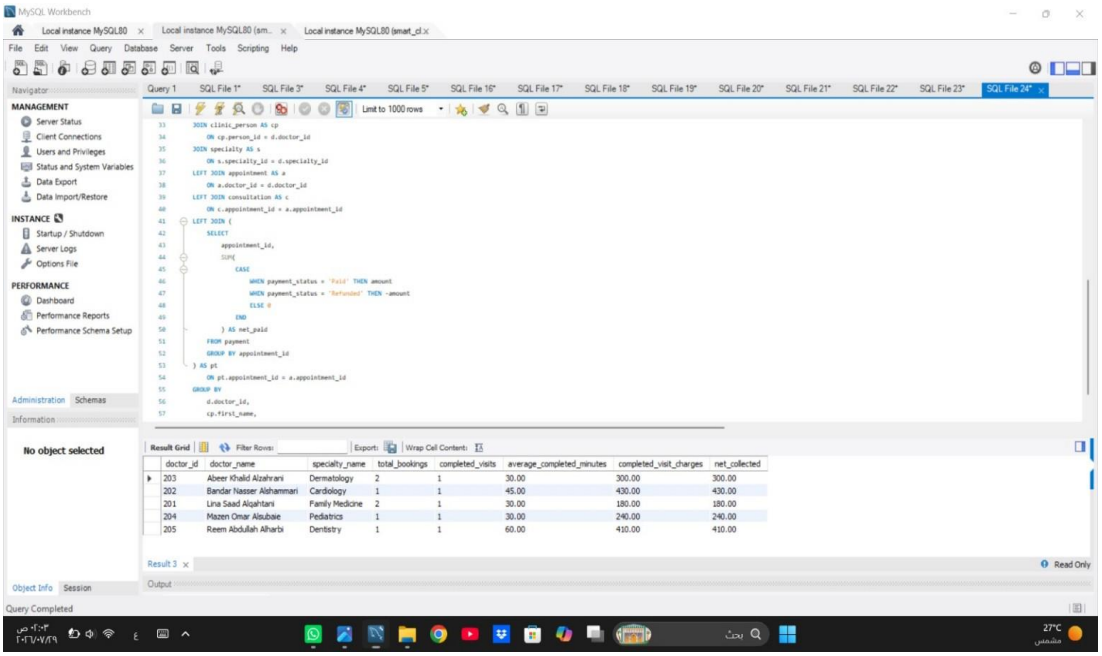

Figure 3.6B. Doctor GROUP BY evidence - Part B and 5-row result.

The final output contains one verified summary row for each of the five doctors.

3.7 UPDATE Statement and Rollback Verification

The UPDATE test temporarily adds an insurance company to patient 105. Variables capture the value before and after the change, while ROLLBACK restores the original record.

```
USE smart_clinic_system;

SET @insurance_before = (
    SELECT COALESCE(insurance_company, 'No insurance')
    FROM patient
    WHERE patient_id = 105
);

START TRANSACTION;

UPDATE patient
SET insurance_company = 'Arabian Shield Cooperative'
WHERE patient_id = 105;

SET @insurance_after = (
    SELECT insurance_company
    FROM patient
    WHERE patient_id = 105
);

ROLLBACK;

SELECT
    105 AS patient_id,
    @insurance_before AS insurance_before_update,
    @insurance_after AS insurance_after_update,
    COALESCE((SELECT insurance_company
              FROM patient
              WHERE patient_id = 105),
             'No insurance') AS insurance_after_rollback;
```

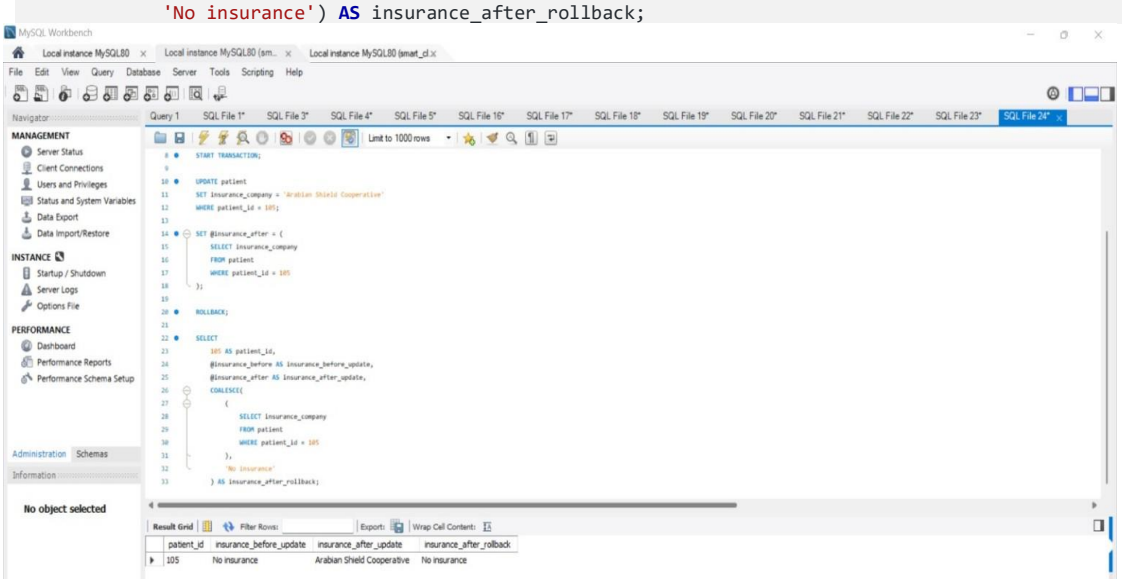

Figure 3.7. UPDATE verification: No insurance → Arabian Shield Cooperative → No insurance.

The one-row verification confirms both the temporary change and successful restoration.

3.8 DELETE Statement and Rollback Verification

The DELETE test removes pending payment 5107 inside a transaction, records the affected-row count, and restores the row with ROLLBACK.

```
USE smart_clinic_system;
 SET @rows_before_delete = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
);

START TRANSACTION;

DELETE FROM payment
WHERE payment_id = 5107
  AND payment_status = 'Pending';

SET @deleted_rows = ROW_COUNT();

SET @rows_after_delete = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
);

ROLLBACK;

SET @rows_after_rollback = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
);

SELECT
    @rows_before_delete AS rows_before_delete,
    @deleted_rows AS deleted_rows,
    @rows_after_delete AS rows_after_delete,
    @rows_after_rollback AS rows_after_rollback;
```

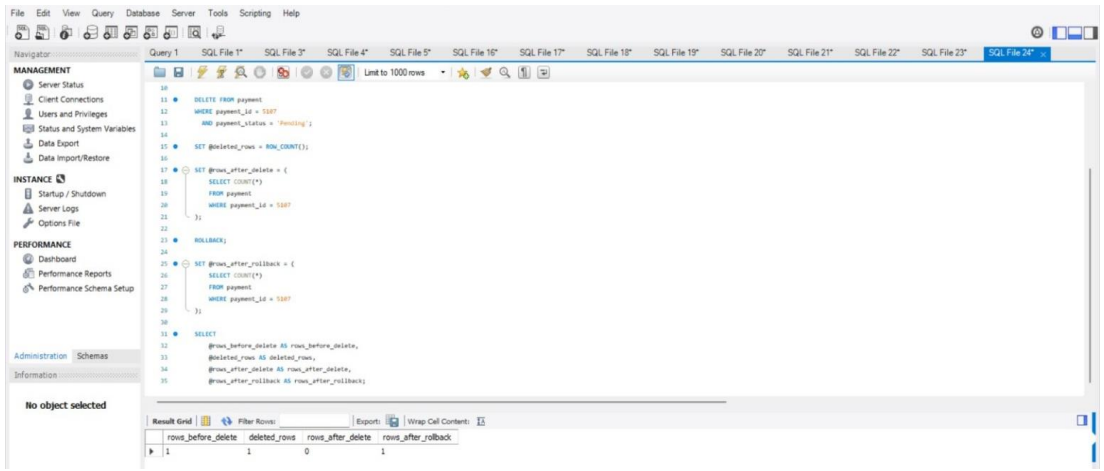

Figure 3.8. DELETE verification result: 1 | 1 | 0 | 1.

The sequence proves that one row existed, one row was deleted, zero remained before rollback, and one row existed again afterward.

3.9 VIEW Result

The reusable vw_visit_financial_status view combines scheduling, patient, doctor, specialty, consultation, and payment information and calculates expected charges, net payments, and outstanding balances.

```
USE smart_clinic_system;

SELECT *
FROM vw_visit_financial_status
ORDER BY starts_at;
```

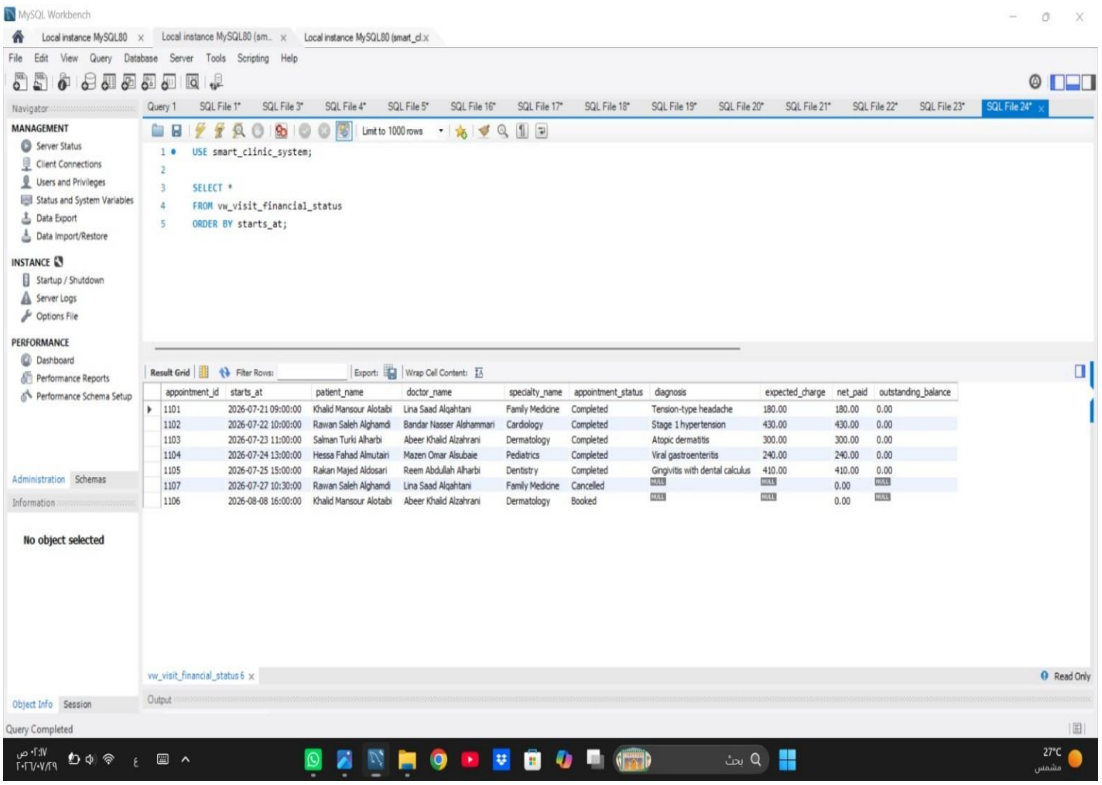

Figure 3.9. vw_visit_financial_status result: 7 appointment rows.

The view returns one financial summary row for every appointment, including appointments without a consultation.

3.10 TRIGGER Creation and Stock-Control Test

The BEFORE INSERT trigger checks medication availability, rejects excessive quantities, and deducts the units dispensed. A temporary prescription item tests the stock change inside a reversible transaction.

```
USE smart_clinic_system;

SET @stock_before_trigger = (
   SELECT stock_on_hand
   FROM medication
   WHERE medication_id = 3102
);

START TRANSACTION;

INSERT INTO prescription
   (prescription_id, consultation_id, issued_at, general_instructions)
VALUES
   (4199, 2104, '2026-07-24 13:45:00',
    'Temporary prescription used to test the trigger.');

INSERT INTO prescription_item
   (prescription_id, medication_id, dose_description,
    frequency_per_day, course_days, units_dispensed,
    administration_route, item_instructions)
VALUES
   (4199, 3102, 'One capsule', 1, 4, 4,
    'Oral', 'Temporary trigger test.');

SET @stock_after_trigger = (
   SELECT stock_on_hand
   FROM medication
   WHERE medication_id = 3102
);

ROLLBACK;

SET @stock_after_rollback = (
   SELECT stock_on_hand
   FROM medication
   WHERE medication_id = 3102
);

SELECT
   @stock_before_trigger AS stock_before_trigger,
   @stock_after_trigger AS stock_after_trigger,
   @stock_after_rollback AS stock_after_rollback;
```

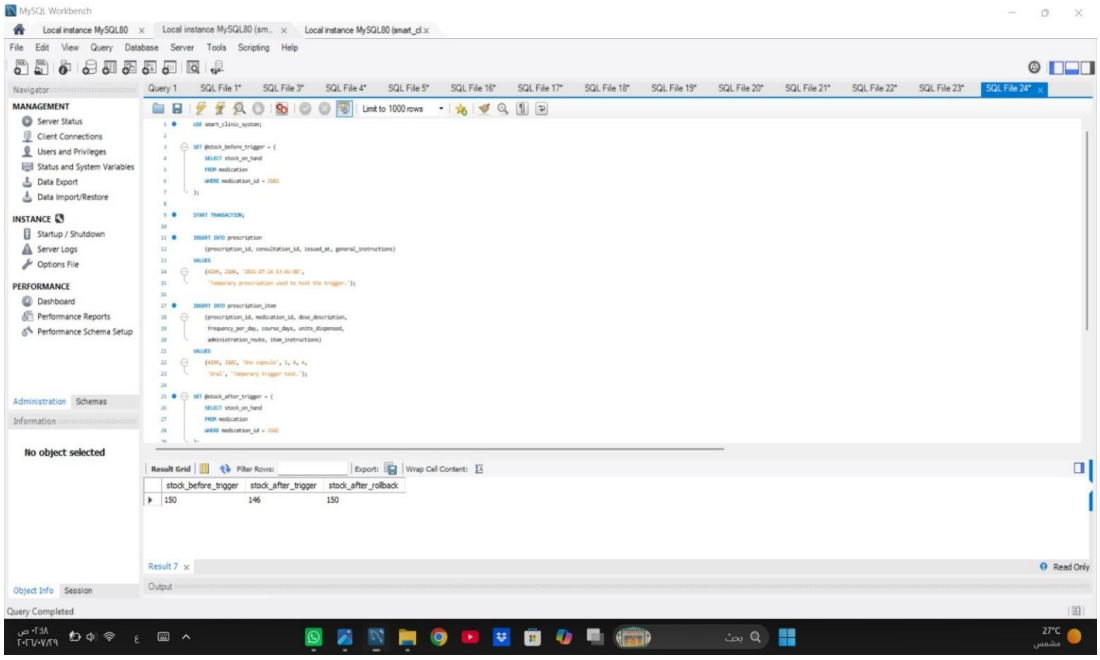

Figure 3.10A. TRIGGER test code - Part A.

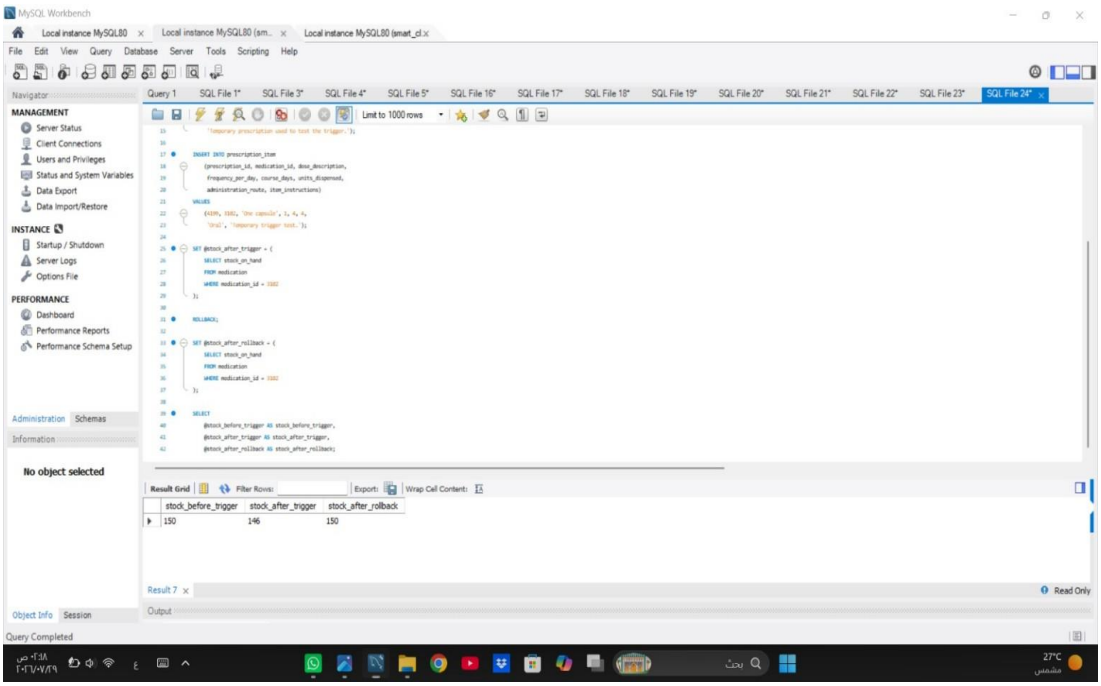

Figure 3.10B. TRIGGER result: stock 150 → 146 → 150 after rollback.

The stock sequence confirms the trigger deduction and the final rollback restoration.

4. Task 4 – Project Reflection

Developing the Smart Clinic Database System gave us a practical opportunity to connect database-design concepts with a realistic clinic workflow. Our most important shared challenge was keeping the EER diagram, relational schema, MySQL implementation, test operations, and screenshot evidence consistent. We identified this issue during review when small differences appeared between the diagram and the script, including attribute lengths, one renamed field, a missing unique-key marker, and unclear cardinality labels. We resolved it by creating a table-by-table checklist, comparing every attribute and relationship with the executable SQL file, correcting the diagram, rerunning the relevant queries, and reviewing each screenshot against its intended requirement.

Sultan Aljohani: As Group Leader and Repository Manager, Sultan coordinated milestones, organized the repository structure, integrated the project files, and completed Task 1. He selected CLINIC_PERSON as a shared supertype specialized into PATIENT and DOCTOR. The alternative was to repeat identity and contact attributes in both subtype tables. Although the selected model requires additional joins when displaying complete records, it reduces redundancy, supports consistent updates, and demonstrates the required total and disjoint specialization. He also reviewed keys, assumptions, relationship directions, and diagram-to-schema consistency.

Amjad Najmi: Amjad implemented the ten MySQL tables, data types, keys, constraints, and linked fictional records for Task 2. His main implementation challenge was satisfying foreign-key dependencies while keeping the sample data realistic. He addressed it by creating and populating parent relations before dependent clinical and financial relations, then checking the populated rows in MySQL Workbench. UNIQUE and CHECK constraints were used to detect duplicated identifiers, invalid appointment durations, incorrect Saudi mobile formats, and negative financial or stock values early.

4. Task 4 – Project Reflection

Abdulaziz Alharbi: Abdulaziz completed Task 3 by preparing and testing the SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER operations. Because data-changing tests could damage the baseline dataset, he chose transaction-based testing with START TRANSACTION and ROLLBACK instead of permanently changing and manually restoring rows. This approach added several verification steps, but it produced repeatable evidence. The final results confirmed the DELETE sequence 1 to 0 to 1 and the trigger stock sequence 150 to 146 to 150.

ChatGPT was used in a limited supporting role for brainstorming document structure, reviewing grammar, and suggesting SQL syntax during troubleshooting. We did not accept its output automatically. Every table definition, query, diagram correction, and result was compared with the project requirements, executed in MySQL Workbench, and modified when the evidence did not match.

If we repeated the project with more time, we would prepare an automated test checklist before implementation, capture evidence during the first verified execution, test the script on more than one MySQL environment, and add stored procedures or audit logging for common clinic operations.

5. Project Artifacts

The following entries provide the final report location, repository, implementation files, diagram sources, and MySQL evidence used for the completed project.

Artifact	Final Entry
Live Report Folder	https://drive.google.com/drive/folders/1arzRO-tMa9YdED1HLBBBPtQnomPO1jSw
GitHub Repository	https://github.com/Database-project-lgtm/Smart-Clinical-Database-System
Complete SQL Script	database/smart_clinic_database.sql
Final EER Diagram	diagrams/er_diagram.png
Editable Diagram Source	diagrams/er_diagram_source.drawio
MySQL Workbench Evidence	evidence/screenshots/ (23 JPG files)
Evidence Index	evidence/Screenshot_Evidence_Index.md

6. Appendix A - Mid-Project Progress Report

SMART CLINIC DATABASE SYSTEM

Mid-Project Progress Report

Stage 1: Database Design and Initial Implementation

Reporting Point: End of Stage 1 - before Task 3

Table 1. Team Members and Primary Contributions

Student Name	ID	Primary Contribution
Sultan Aljohani	**240035689**	Group Leader and Repository Manager, Task 1: Database Design
Amjad Najmi	**240020509**	Task 2: Database Implementation
Abdulaziz Alharbi	**250021379**	Task 3: SQL Operations

❖ Progress Summary

At this reporting point, the project has reached the end of Stage 1. The team has defined the system scope, allocated responsibilities, completed the database design, and prepared the core MySQL implementation for validation. The design maps ten related tables and includes the required EER specialization, integrity constraints, and fictional Saudi-context data. Full MySQL Workbench verification, Task 2 screenshot evidence, Task 3 SQL operations, and the Project Reflection remain scheduled for the next stage.

6. Appendix A - Mid-Project Progress Report

❖ Midpoint Progress Status

Table 2. Stage 1 Workstream Status

Workstream	Status	Evidence / Expected Output
Project planning and responsibility allocation	**Completed**	Contribution record, approved repository structure, and delivery plan.
Task 1 Database Design	**Completed and reviewed**	Ten-relation EER model, relational schema, keys, cardinalities, assumptions, and business rules.
Task 2 Core Implementation	**Prepared for validation**	MySQL database setup, ten CREATE TABLE statements, constraints, and fictional sample data.
Task 2 Workbench Evidence	**Planned for next stage**	Successful execution, populated-table results, and readable screenshots.
Task 3 SQL Operations	**Planned for next stage**	SELECT, JOIN, nested query, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER.
Task 4 Project Reflection	**Planned for next stage**	Write Reflection after all technical work and evidence are complete.

❖ Work Completed the Midpoint

- ✓ Defined ten connected relations: CLINIC_PERSON, PATIENT, SPECIALTY, DOCTOR, APPOINTMENT, CONSULTATION, MEDICATION, PRESCRIPTION, PRESCRIPTION_ITEM, and PAYMENT.
- ✓ Modeled CLINIC_PERSON as the supertype and PATIENT and DOCTOR as total, disjoint subtypes.
- ✓ Specified primary keys, foreign keys, unique keys, CHECK constraints, and controlled ENUM values.
- ✓ Mapped relationships and optional cardinalities between appointments, consultations, prescriptions, medication items, and payments.
- ✓ Prepared a repeatable MySQL 8.0 script using utf8mb4 and a dependency-aware table creation and insertion order.
- ✓ Conducted an initial consistency review between the EER design, relational schema, and core SQL implementation.

6. Appendix A - Mid-Project Progress Report

❖ Key Design Decisions

Table 3. Design Decisions and Rationale

Decision	Reason
Shared CLINIC_PERSON supertype	Avoids repeating identity and contact attributes in separate patient and doctor tables.
Total, disjoint specialization	Gives each clinic person one clear subtype while demonstrating the required EER feature.
Separate SPECIALTY relation	Normalizes specialty details and lets multiple doctors share one controlled specialty record.
Separate APPOINTMENT and CONSULTATION	Distinguishes planned visits from clinical outcomes and allows an appointment to exist before consultation.
Associative PRESCRIPTION_ITEM relation	Resolves the prescription-medication many-to-many relationship while storing dosage, route, duration, and quantity.
Multiple PAYMENT records per appointment	Supports installments, insurance processing, refunds, and different payment statuses.
One evolving SQL file	Keeps the design, implementation, later operations, VIEW, and TRIGGER in one controlled project source.

❖ Challenges and Responses

Table 4. Stage 1 Challenges and Responses

Stage 1 Challenge	Response
Aligning the EER model with the relational schema	Cross-checked every entity, key, relationship, and cardinality against the SQL design before implementation testing.
Maintaining referential insertion order	Ordered parent tables before dependent clinical and financial tables so every foreign key can be satisfied.
Preventing schedule conflicts	Added unique doctor-time and patient-time constraints to APPOINTMENT.

6. Appendix A - Mid-Project Progress Report

Representing optional clinical events	Separated APPOINTMENT, CONSULTATION, and PRESCRIPTION so records are created only when each real event occurs.
Maintaining realistic and valid sample data	Used linked Saudi-context records, unique identifiers, +966 validation rules, and CHECK constraints.

❖ Team Contributions at the Midpoint

Table 5. Recorded Stage 1 Contributions

Member	Midpoint Contribution	Status	Output
Sultan Aljohani	Team coordination, repository planning, requirements, entities, keys, relationships, cardinalities, EER specialization, and schema alignment.	**Completed and reviewed**	Task 1 design package and repository organization.
Amjad Najmi	Database creation, ten table definitions, data types, constraints, insertion order, and fictional sample records.	**Prepared for validation**	Core smart_clinic_database.sql implementation.
Abdulaziz Alharbi	Reviewed Task 3 requirements and prepared the operation, testing, and screenshot checklist for the next stage.	**Planned for next stage**	Task 3 work and evidence plan.
All members	Cross-review of Stage 1, design-implementation consistency checking, milestone planning, and preparation of this progress report.	**In progress**	Review notes and Mid-Project Progress Report.

6. Appendix A - Mid-Project Progress Report

❖ Group's Plan for Next-Stage.

1) Review and approve the complete Stage 1 EER model, relational schema, assumptions, constraints, and sample data as a team.
2) Run the complete Stage 1 script in MySQL Workbench and correct any environment-specific execution issues.
3) Capture and organize the required Task 2 screenshots showing the populated tables and successful execution.
4) Implement and test Task 3 SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER operations in the same SQL file.
5) Capture Task 3 code-and-result evidence and connect each image to the Screenshot Evidence Index.
6) Write the Project Reflection after the technical work, results, and team contributions have been verified.
7) Complete the README and final report, then verify all file names, links, Word/PDF outputs, and submission requirements.

7. Appendix B - Complete SQL Implementation

database/smart_clinic_database.sql

```
/*
================================================================================
Smart Clinic Database System
IT244 - Database Design and Implementation Project

Project Team
1. Sultan Aljohani     - 240035689 - Group Leader, Repository Manager,
                                     and Task 1: Database Design
2. Amjad Najmi         - 240020509 - Task 2: Database Implementation
3. Abdulaziz Alharbi   - 250021379 - Task 3: SQL Operations

This MySQL 8.0 script contains the relational implementation required for
Tasks 1-3. The project reflection is prepared jointly in the final report.
All names, identifiers, contact details, and medical records are fictional.
================================================================================
*/

-- ============================================================================
-- PART A - DATABASE SETUP AND RELATIONAL DESIGN
-- ============================================================================

DROP DATABASE IF EXISTS smart_clinic_system;

CREATE DATABASE smart_clinic_system
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci;

USE smart_clinic_system;
SET NAMES utf8mb4;

/*
EER design summary
------------------
CLINIC_PERSON is the supertype for shared identity and contact attributes.
PATIENT and DOCTOR are total, disjoint subtypes at the application level.

Main relationships
------------------
SPECIALTY      1 ---- M DOCTOR
PATIENT        1 ---- M APPOINTMENT
DOCTOR         1 ---- M APPOINTMENT
APPOINTMENT    1 ---- 0..1 CONSULTATION
CONSULTATION   1 ---- M PRESCRIPTION
PRESCRIPTION   1 ---- M PRESCRIPTION_ITEM
MEDICATION     1 ---- M PRESCRIPTION_ITEM
APPOINTMENT    1 ---- M PAYMENT

PRESCRIPTION_ITEM resolves the many-to-many relationship between
PRESCRIPTION and MEDICATION while storing the dosage and course details.
*/
```

7. Appendix B - Complete SQL Implementation

```sql
CREATE TABLE clinic_person (
    person_id             INT           NOT NULL,
    national_identifier   CHAR(10)      NOT NULL,
    first_name            VARCHAR(45)   NOT NULL,
    last_name             VARCHAR(65)   NOT NULL,
    birth_date            DATE          NOT NULL,
    sex                   ENUM('Female', 'Male') NOT NULL,
    mobile_number         VARCHAR(13)   NOT NULL,
    email_address         VARCHAR(120)  NULL,
    city                  VARCHAR(60)   NOT NULL,
    CONSTRAINT pk_clinic_person PRIMARY KEY (person_id),
    CONSTRAINT uq_clinic_person_national UNIQUE (national_identifier),
    CONSTRAINT uq_clinic_person_mobile UNIQUE (mobile_number),
    CONSTRAINT uq_clinic_person_email UNIQUE (email_address),
    CONSTRAINT ck_clinic_person_national
        CHECK (national_identifier REGEXP '^[12][0-9]{9}$'),
    CONSTRAINT ck_clinic_person_mobile
        CHECK (mobile_number REGEXP '^\\+9665[0-9]{8}$')
) ENGINE = InnoDB;

CREATE TABLE patient (
    patient_id            INT           NOT NULL,
    medical_record_no     VARCHAR(12)   NOT NULL,
    blood_group           ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')
NOT NULL,
    insurance_company     VARCHAR(100)  NULL,
    emergency_contact_name VARCHAR(100) NOT NULL,
    emergency_mobile      VARCHAR(13)   NOT NULL,
    registered_on         DATE          NOT NULL,
    CONSTRAINT pk_patient PRIMARY KEY (patient_id),
    CONSTRAINT uq_patient_medical_record UNIQUE (medical_record_no),
    CONSTRAINT fk_patient_person
        FOREIGN KEY (patient_id) REFERENCES clinic_person (person_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_patient_emergency_mobile
        CHECK (emergency_mobile REGEXP '^\\+9665[0-9]{8}$')
) ENGINE = InnoDB;

CREATE TABLE specialty (
    specialty_id          SMALLINT      NOT NULL,
    specialty_name        VARCHAR(80)   NOT NULL,
    clinic_zone           VARCHAR(20)   NOT NULL,
    standard_slot_minutes SMALLINT      NOT NULL DEFAULT 30,
    CONSTRAINT pk_specialty PRIMARY KEY (specialty_id),
    CONSTRAINT uq_specialty_name UNIQUE (specialty_name),
    CONSTRAINT ck_specialty_slot
        CHECK (standard_slot_minutes BETWEEN 15 AND 120)
) ENGINE = InnoDB;

CREATE TABLE doctor (
    doctor_id             INT           NOT NULL,
    specialty_id          SMALLINT      NOT NULL,
    professional_license  VARCHAR(25)   NOT NULL,
```

7. Appendix B - Complete SQL Implementation

```
    office_code            VARCHAR(12)    NOT NULL,
    hired_on               DATE           NOT NULL,
    consultation_fee       DECIMAL(9, 2)  NOT NULL,
    availability_status    ENUM('Active', 'Leave', 'Inactive') NOT NULL DEFAULT
'Active',
    CONSTRAINT pk_doctor PRIMARY KEY (doctor_id),
    CONSTRAINT uq_doctor_license UNIQUE (professional_license),
    CONSTRAINT uq_doctor_office UNIQUE (office_code),
    CONSTRAINT fk_doctor_person
        FOREIGN KEY (doctor_id) REFERENCES clinic_person (person_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_doctor_specialty
        FOREIGN KEY (specialty_id) REFERENCES specialty (specialty_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_doctor_fee CHECK (consultation_fee >= 0)
) ENGINE = InnoDB;

CREATE TABLE appointment (
    appointment_id         INT            NOT NULL,
    patient_id             INT            NOT NULL,
    doctor_id              INT            NOT NULL,
    starts_at              DATETIME       NOT NULL,
    planned_minutes        SMALLINT       NOT NULL,
    appointment_status     ENUM('Booked', 'Completed', 'Cancelled', 'No Show') NOT
NULL DEFAULT 'Booked',
    booking_source         ENUM('Reception', 'Phone', 'Portal', 'Walk-in') NOT
NULL,
    visit_reason           VARCHAR(250)   NOT NULL,
    booked_at              TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_appointment PRIMARY KEY (appointment_id),
    CONSTRAINT uq_appointment_doctor_time UNIQUE (doctor_id, starts_at),
    CONSTRAINT uq_appointment_patient_time UNIQUE (patient_id, starts_at),
    CONSTRAINT fk_appointment_patient
        FOREIGN KEY (patient_id) REFERENCES patient (patient_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_appointment_doctor
        FOREIGN KEY (doctor_id) REFERENCES doctor (doctor_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_appointment_minutes
        CHECK (planned_minutes BETWEEN 15 AND 120)
) ENGINE = InnoDB;

CREATE TABLE consultation (
    consultation_id        INT            NOT NULL,
    appointment_id         INT            NOT NULL,
    consulted_at           DATETIME       NOT NULL,
    diagnosis              VARCHAR(220)   NOT NULL,
    assessment_notes       VARCHAR(600)   NOT NULL,
    service_fee            DECIMAL(9, 2)  NOT NULL DEFAULT 0.00,
    follow_up_date         DATE           NULL,
    CONSTRAINT pk_consultation PRIMARY KEY (consultation_id),
```

7. Appendix B - Complete SQL Implementation

```
    CONSTRAINT uq_consultation_appointment UNIQUE (appointment_id),
    CONSTRAINT fk_consultation_appointment
        FOREIGN KEY (appointment_id) REFERENCES appointment (appointment_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_consultation_service_fee CHECK (service_fee >= 0)
) ENGINE = InnoDB;

CREATE TABLE medication (
    medication_id          INT            NOT NULL,
    generic_name           VARCHAR(100)   NOT NULL,
    brand_name             VARCHAR(100)   NOT NULL,
    strength               VARCHAR(35)    NOT NULL,
    dosage_form            ENUM('Tablet', 'Capsule', 'Cream', 'Sachet',
'Suspension', 'Mouthwash', 'Other') NOT NULL,
    unit_price             DECIMAL(9, 2)  NOT NULL,
    stock_on_hand          INT            NOT NULL,
    reorder_level          INT            NOT NULL,
    requires_prescription  BOOLEAN        NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_medication PRIMARY KEY (medication_id),
    CONSTRAINT uq_medication_product
        UNIQUE (brand_name, strength, dosage_form),
    CONSTRAINT ck_medication_price CHECK (unit_price >= 0),
    CONSTRAINT ck_medication_stock CHECK (stock_on_hand >= 0),
    CONSTRAINT ck_medication_reorder CHECK (reorder_level >= 0)
) ENGINE = InnoDB;

CREATE TABLE prescription (
    prescription_id        INT            NOT NULL,
    consultation_id        INT            NOT NULL,
    issued_at              DATETIME       NOT NULL,
    general_instructions   VARCHAR(400)   NULL,
    CONSTRAINT pk_prescription PRIMARY KEY (prescription_id),
    CONSTRAINT fk_prescription_consultation
        FOREIGN KEY (consultation_id) REFERENCES consultation (consultation_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE prescription_item (
    prescription_id        INT            NOT NULL,
    medication_id          INT            NOT NULL,
    dose_description       VARCHAR(100)   NOT NULL,
    frequency_per_day      TINYINT        NOT NULL,
    course_days            SMALLINT       NOT NULL,
    units_dispensed        SMALLINT       NOT NULL,
    administration_route   ENUM('Oral', 'Topical', 'Dental', 'Other') NOT NULL,
    item_instructions      VARCHAR(300)   NULL,
    CONSTRAINT pk_prescription_item
        PRIMARY KEY (prescription_id, medication_id),
    CONSTRAINT fk_prescription_item_header
        FOREIGN KEY (prescription_id) REFERENCES prescription (prescription_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_prescription_item_medication
```

7. Appendix B - Complete SQL Implementation

```
        FOREIGN KEY (medication_id) REFERENCES medication (medication_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_prescription_item_frequency CHECK (frequency_per_day BETWEEN 1
AND 12),
    CONSTRAINT ck_prescription_item_course CHECK (course_days > 0),
    CONSTRAINT ck_prescription_item_units CHECK (units_dispensed > 0)
) ENGINE = InnoDB;

CREATE TABLE payment (
    payment_id             INT            NOT NULL,
    appointment_id         INT            NOT NULL,
    amount                 DECIMAL(9, 2)  NOT NULL,
    recorded_at            DATETIME       NOT NULL,
    payment_method         ENUM('Cash', 'Card', 'Bank Transfer', 'Insurance') NOT
NULL,
    payment_status         ENUM('Pending', 'Paid', 'Refunded', 'Failed') NOT NULL
DEFAULT 'Pending',
    payer_type             ENUM('Patient', 'Insurance') NOT NULL,
    receipt_number         VARCHAR(35)    NULL,
    CONSTRAINT pk_payment PRIMARY KEY (payment_id),
    CONSTRAINT uq_payment_receipt UNIQUE (receipt_number),
    CONSTRAINT fk_payment_appointment
        FOREIGN KEY (appointment_id) REFERENCES appointment (appointment_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_payment_amount CHECK (amount > 0)
) ENGINE = InnoDB;

-- ================================================================================
-- PART B - FICTIONAL SAMPLE DATA
-- Every principal table contains at least five records.
-- ================================================================================

START TRANSACTION;

INSERT INTO clinic_person
    (person_id, national_identifier, first_name, last_name, birth_date, sex,
     mobile_number, email_address, city)
VALUES
    (101, '1084512376', 'Khalid Mansour', 'Alotaibi',    '1991-05-14', 'Male',
'+966561234501', 'khalid.alotaibi@example.sa',  'Riyadh'),
    (102, '1073625148', 'Rawan Saleh',    'Alghamdi',    '1988-11-02', 'Female',
'+966561234502', 'rawan.alghamdi@example.sa',    'Riyadh'),
    (103, '1062748351', 'Salman Turki',   'Alharbi',     '1997-03-26', 'Male',
'+966561234503', 'salman.alharbi@example.sa',     'Al Kharj'),
    (104, '1165832409', 'Hessa Fahad',    'Almutairi',   '2016-08-19', 'Female',
'+966561234504', 'hessa.almutairi@example.sa',    'Riyadh'),
    (105, '1051947283', 'Rakan Majed',    'Aldosari',    '1983-12-07', 'Male',
'+966561234505', 'rakan.aldosari@example.sa',     'Diriyah'),
    (201, '1048372615', 'Lina Saad',      'Alqahtani',   '1980-04-10', 'Female',
'+966569876501', 'lina.alqahtani@clinic.example.sa', 'Riyadh'),
    (202, '1037461928', 'Bandar Nasser',  'Alshammari',  '1976-09-21', 'Male',
'+966569876502', 'bandar.alshammari@clinic.example.sa', 'Riyadh'),
    (203, '1026583741', 'Abeer Khalid',   'Alzahrani',   '1985-01-16', 'Female',
```

7. Appendix B - Complete SQL Implementation

```
'+966569876503', 'abeer.alzahrani@clinic.example.sa', 'Riyadh'),
    (204, '1019274658', 'Mazen Omar',     'Alsubaie',    '1979-06-30', 'Male',
'+966569876504', 'mazen.alsubaie@clinic.example.sa', 'Riyadh'),
    (205, '1092847365', 'Reem Abdullah',  'Alharbi',     '1984-02-08', 'Female',
'+966569876505', 'reem.alharbi@clinic.example.sa', 'Riyadh');

INSERT INTO patient
    (patient_id, medical_record_no, blood_group, insurance_company,
     emergency_contact_name, emergency_mobile, registered_on)
VALUES
    (101, 'MR-260101', 'O+',  'Bupa Arabia',       'Mansour Alotaibi',
'+966551110101', '2026-06-02'),
    (102, 'MR-260102', 'A-',  'Tawuniya',          'Saleh Alghamdi',
'+966551110102', '2026-06-05'),
    (103, 'MR-260103', 'B+',  'MedGulf',           'Turki Alharbi',
'+966551110103', '2026-06-09'),
    (104, 'MR-260104', 'O-',  'Al Rajhi Takaful',  'Fahad Almutairi',
'+966551110104', '2026-06-12'),
    (105, 'MR-260105', 'AB+', NULL,                'Majed Aldosari',
'+966551110105', '2026-06-15');

INSERT INTO specialty
    (specialty_id, specialty_name, clinic_zone, standard_slot_minutes)
VALUES
    (11, 'Family Medicine', 'Ground-G1', 30),
    (22, 'Cardiology',      'First-C1',  45),
    (33, 'Dermatology',     'First-D2',  30),
    (44, 'Pediatrics',      'Second-P1', 30),
    (55, 'Dentistry',       'Second-D3', 60);

INSERT INTO doctor
    (doctor_id, specialty_id, professional_license, office_code, hired_on,
     consultation_fee, availability_status)
VALUES
    (201, 11, 'SCFHS-FM-2601', 'G-104', '2020-03-15', 140.00, 'Active'),
    (202, 22, 'SCFHS-CR-2602', 'C-208', '2018-10-01', 320.00, 'Active'),
    (203, 33, 'SCFHS-DM-2603', 'D-212', '2021-05-20', 240.00, 'Active'),
    (204, 44, 'SCFHS-PD-2604', 'P-305', '2019-01-12', 190.00, 'Active'),
    (205, 55, 'SCFHS-DN-2605', 'D-318', '2022-08-08', 230.00, 'Active');

INSERT INTO appointment
    (appointment_id, patient_id, doctor_id, starts_at, planned_minutes,
     appointment_status, booking_source, visit_reason)
VALUES
    (1101, 101, 201, '2026-07-21 09:00:00', 30, 'Completed', 'Portal',
'Recurring headache and neck muscle tension'),
    (1102, 102, 202, '2026-07-22 10:00:00', 45, 'Completed', 'Reception', 'Elevated
home blood pressure readings'),
    (1103, 103, 203, '2026-07-23 11:00:00', 30, 'Completed', 'Phone',     'Dry and
inflamed skin on both forearms'),
    (1104, 104, 204, '2026-07-24 13:00:00', 30, 'Completed', 'Reception',
'Vomiting, diarrhea, and reduced appetite'),
    (1105, 105, 205, '2026-07-25 15:00:00', 60, 'Completed', 'Walk-in',   'Bleeding
gums and persistent bad breath'),
    (1106, 101, 203, '2026-08-08 16:00:00', 30, 'Booked',    'Portal',    'Follow-
```

7. Appendix B - Complete SQL Implementation

```
up assessment for a new skin lesion'),
    (1107, 102, 201, '2026-07-27 10:30:00', 30, 'Cancelled', 'Phone',     'General
fatigue and sleep disturbance');

INSERT INTO consultation
    (consultation_id, appointment_id, consulted_at, diagnosis,
     assessment_notes, service_fee, follow_up_date)
VALUES
    (2101, 1101, '2026-07-21 09:35:00', 'Tension-type headache',
     'Neurological screening was normal. Hydration, posture correction, and short-
term analgesia were advised.', 40.00, '2026-08-04'),
    (2102, 1102, '2026-07-22 10:50:00', 'Stage 1 hypertension',
     'Medication was initiated with home pressure monitoring and reduced dietary
sodium.', 110.00, '2026-08-19'),
    (2103, 1103, '2026-07-23 11:35:00', 'Atopic dermatitis',
     'The patient was advised to avoid fragranced products and use a short topical
treatment course.', 60.00, '2026-08-06'),
    (2104, 1104, '2026-07-24 13:35:00', 'Viral gastroenteritis',
     'Oral rehydration, light meals, temperature observation, and warning-sign
education were provided.', 50.00, NULL),
    (2105, 1105, '2026-07-25 16:05:00', 'Gingivitis with dental calculus',
     'Professional cleaning was completed and oral hygiene instructions were
demonstrated.', 180.00, '2026-08-22');

INSERT INTO medication
    (medication_id, generic_name, brand_name, strength, dosage_form, unit_price,
     stock_on_hand, reorder_level, requires_prescription)
VALUES
    (3101, 'Ibuprofen',              'Brufen',          '400 mg',       'Tablet',
1.10,  70, 80, FALSE),
    (3102, 'Azithromycin',           'Zithromax',       '250 mg',       'Capsule',
4.50, 150, 50, TRUE),
    (3103, 'Losartan',               'Cozaar',           '50 mg',        'Tablet',
2.20,  65, 70, TRUE),
    (3104, 'Mometasone furoate',     'Elocon',           '0.1%',         'Cream',
22.00,  35, 40, TRUE),
    (3105, 'Oral rehydration salts', 'Hydralyte',        '20.5 g',       'Sachet',
3.75, 120, 60, FALSE),
    (3106, 'Chlorhexidine',          'Corsodyl',         '0.2%',
'Mouthwash', 18.00,  45, 50, FALSE),
    (3107, 'Paracetamol',            'Panadol Children', '120 mg/5 mL',
'Suspension', 16.50,  90, 50, FALSE);

INSERT INTO prescription
    (prescription_id, consultation_id, issued_at, general_instructions)
VALUES
    (4101, 2101, '2026-07-21 09:38:00', 'Use only when the headache interferes with
daily activity.'),
    (4102, 2102, '2026-07-22 10:53:00', 'Measure blood pressure at home and record
the readings.'),
    (4103, 2103, '2026-07-23 11:38:00', 'Stop use and contact the clinic if
irritation becomes worse.'),
    (4104, 2104, '2026-07-24 13:38:00', 'Give frequent fluids and monitor for signs
of dehydration.'),
    (4105, 2105, '2026-07-25 16:08:00', 'Continue brushing and flossing carefully
```

7. Appendix B - Complete SQL Implementation

```
during treatment.');

INSERT INTO prescription_item
    (prescription_id, medication_id, dose_description, frequency_per_day,
     course_days, units_dispensed, administration_route, item_instructions)
VALUES
    (4101, 3101, 'One tablet after food',       2,  5, 10, 'Oral',    'Do not take
on an empty stomach.'),
    (4102, 3103, 'One tablet',                  1, 30, 30, 'Oral',    'Take at the
same time every day.'),
    (4103, 3104, 'Apply a thin layer',          2,  7,  1, 'Topical', 'Apply only
to affected skin.'),
    (4104, 3105, 'One sachet in clean water',   3,  3,  9, 'Oral',    'Prepare a
fresh solution each time.'),
    (4104, 3107, 'Ten milliliters when needed', 3,  3,  1, 'Oral',    'Use the
supplied measuring device.'),
    (4105, 3106, 'Rinse with 15 milliliters',   2,  7,  1, 'Dental',  'Do not
swallow the mouthwash.'),
    (4105, 3101, 'One tablet after food',       2,  3,  6, 'Oral',    'Use only for
post-cleaning discomfort.');

INSERT INTO payment
    (payment_id, appointment_id, amount, recorded_at, payment_method,
     payment_status, payer_type, receipt_number)
VALUES
    (5101, 1101, 180.00, '2026-07-21 09:45:00', 'Cash',          'Paid',
'Patient',   'RC-260721-01'),
    (5102, 1102, 250.00, '2026-07-22 10:55:00', 'Insurance',     'Paid',
'Insurance', 'RC-260722-01'),
    (5103, 1102, 180.00, '2026-07-22 10:57:00', 'Card',          'Paid',
'Patient',   'RC-260722-02'),
    (5104, 1103, 300.00, '2026-07-23 11:42:00', 'Card',          'Paid',
'Patient',   'RC-260723-01'),
    (5105, 1104, 240.00, '2026-07-24 13:42:00', 'Insurance',     'Paid',
'Insurance', 'RC-260724-01'),
    (5106, 1105, 410.00, '2026-07-25 16:12:00', 'Bank Transfer', 'Paid',
'Patient',   'RC-260725-01'),
    (5107, 1106,  50.00, '2026-08-03 14:10:00', 'Card',          'Pending',
'Patient',   NULL);

COMMIT;

-- ============================================================================
-- PART C - TASK 2 VERIFICATION AND SCREENSHOT QUERIES
-- Run each statement separately so the code and its result appear together.
-- ============================================================================

SHOW TABLES FROM smart_clinic_system;

SELECT 'clinic_person' AS table_name, COUNT(*) AS row_count FROM clinic_person
UNION ALL SELECT 'patient', COUNT(*) FROM patient
UNION ALL SELECT 'specialty', COUNT(*) FROM specialty
UNION ALL SELECT 'doctor', COUNT(*) FROM doctor
UNION ALL SELECT 'appointment', COUNT(*) FROM appointment
UNION ALL SELECT 'consultation', COUNT(*) FROM consultation
```

7. Appendix B - Complete SQL Implementation

```
UNION ALL SELECT 'medication', COUNT(*) FROM medication
UNION ALL SELECT 'prescription', COUNT(*) FROM prescription
UNION ALL SELECT 'prescription_item', COUNT(*) FROM prescription_item
UNION ALL SELECT 'payment', COUNT(*) FROM payment;

SELECT * FROM clinic_person ORDER BY person_id;
SELECT * FROM patient ORDER BY patient_id;
SELECT * FROM specialty ORDER BY specialty_id;
SELECT * FROM doctor ORDER BY doctor_id;
SELECT * FROM appointment ORDER BY starts_at;
SELECT * FROM consultation ORDER BY consultation_id;
SELECT * FROM medication ORDER BY medication_id;
SELECT * FROM prescription ORDER BY prescription_id;
SELECT * FROM prescription_item ORDER BY prescription_id, medication_id;
SELECT * FROM payment ORDER BY payment_id;

-- ============================================================================
-- PART D - TASK 3 SQL OPERATIONS
-- Each assessed operation has a purpose statement and a verifiable result.
-- ============================================================================

-- ----------------------------------------------------------------------------
-- TASK 3.1A - SELECT STATEMENT
-- Purpose: Display completed appointments in chronological order for the
-- clinic's completed-visit register.
-- ----------------------------------------------------------------------------
SELECT
    appointment_id,
    starts_at,
    planned_minutes,
    booking_source,
    visit_reason
FROM appointment
WHERE appointment_status = 'Completed'
ORDER BY starts_at;

-- ----------------------------------------------------------------------------
-- TASK 3.1B - SELECT STATEMENT
-- Purpose: Identify products that have reached or fallen below their reorder
-- thresholds so the clinic can prioritize replenishment.
-- ----------------------------------------------------------------------------
SELECT
    medication_id,
    brand_name,
    strength,
    stock_on_hand,
    reorder_level,
    (reorder_level - stock_on_hand) AS units_below_target
FROM medication
WHERE stock_on_hand <= reorder_level
ORDER BY stock_on_hand, brand_name;

-- ----------------------------------------------------------------------------
-- TASK 3.2A - JOIN QUERY
-- Purpose: Produce a readable appointment schedule containing the patient,
```

7. Appendix B - Complete SQL Implementation

```sql
-- doctor, specialty, booking status, and appointment time.
-- ------------------------------------------------------------------------------
SELECT
    a.appointment_id,
    a.starts_at,
    CONCAT(pp.first_name, ' ', pp.last_name) AS patient_name,
    CONCAT(dp.first_name, ' ', dp.last_name) AS doctor_name,
    s.specialty_name,
    a.appointment_status
FROM appointment AS a
JOIN patient AS p
    ON p.patient_id = a.patient_id
JOIN clinic_person AS pp
    ON pp.person_id = p.patient_id
JOIN doctor AS d
    ON d.doctor_id = a.doctor_id
JOIN clinic_person AS dp
    ON dp.person_id = d.doctor_id
JOIN specialty AS s
    ON s.specialty_id = d.specialty_id
ORDER BY a.starts_at;

-- ------------------------------------------------------------------------------
-- TASK 3.2B - JOIN QUERY
-- Purpose: Connect patients and diagnoses with the medicines issued to them,
-- including dosage, course length, and administration route.
-- ------------------------------------------------------------------------------
SELECT
    pat.medical_record_no,
    CONCAT(cp.first_name, ' ', cp.last_name) AS patient_name,
    c.diagnosis,
    m.brand_name AS medication,
    m.strength,
    pi.dose_description,
    pi.frequency_per_day,
    pi.course_days,
    pi.administration_route
FROM patient AS pat
JOIN clinic_person AS cp
    ON cp.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN consultation AS c
    ON c.appointment_id = a.appointment_id
JOIN prescription AS pr
    ON pr.consultation_id = c.consultation_id
JOIN prescription_item AS pi
    ON pi.prescription_id = pr.prescription_id
JOIN medication AS m
    ON m.medication_id = pi.medication_id
ORDER BY patient_name, medication;

-- ------------------------------------------------------------------------------
-- TASK 3.3 - NESTED QUERY
-- Purpose: Find patients whose total completed payments are higher than the
```

7. Appendix B - Complete SQL Implementation

```sql
-- average total paid per paying patient, rather than the average single payment.
-- ------------------------------------------------------------------------------
SELECT
    pat.patient_id,
    pat.medical_record_no,
    CONCAT(cp.first_name, ' ', cp.last_name) AS patient_name,
    SUM(pay.amount) AS patient_paid_total
FROM patient AS pat
JOIN clinic_person AS cp
    ON cp.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
WHERE pay.payment_status = 'Paid'
GROUP BY pat.patient_id, pat.medical_record_no, cp.first_name, cp.last_name
HAVING SUM(pay.amount) > (
    SELECT AVG(patient_total)
    FROM (
        SELECT
            a2.patient_id,
            SUM(pay2.amount) AS patient_total
        FROM appointment AS a2
        JOIN payment AS pay2
            ON pay2.appointment_id = a2.appointment_id
        WHERE pay2.payment_status = 'Paid'
        GROUP BY a2.patient_id
    ) AS paid_by_patient
)
ORDER BY patient_paid_total DESC;

-- ------------------------------------------------------------------------------
-- TASK 3.4 - AGGREGATE FUNCTIONS WITH GROUP BY
-- Purpose: Summarize each doctor's bookings, completed visits, average visit
-- length, completed-visit charges, and collected payments. Payment totals are
-- aggregated by appointment before the main join to avoid duplicated charges.
-- ------------------------------------------------------------------------------
SELECT
    d.doctor_id,
    CONCAT(cp.first_name, ' ', cp.last_name) AS doctor_name,
    s.specialty_name,
    COUNT(a.appointment_id) AS total_bookings,
    SUM(CASE WHEN a.appointment_status = 'Completed' THEN 1 ELSE 0 END)
        AS completed_visits,
    ROUND(AVG(CASE WHEN a.appointment_status = 'Completed'
                   THEN a.planned_minutes END), 2)
        AS average_completed_minutes,
    COALESCE(SUM(CASE WHEN a.appointment_status = 'Completed'
                      THEN d.consultation_fee + COALESCE(c.service_fee, 0)
                      ELSE 0 END), 0.00)
        AS completed_visit_charges,
    COALESCE(SUM(pt.net_paid), 0.00) AS net_collected
FROM doctor AS d
JOIN clinic_person AS cp
    ON cp.person_id = d.doctor_id
```

7. Appendix B - Complete SQL Implementation

```sql
JOIN specialty AS s
    ON s.specialty_id = d.specialty_id
LEFT JOIN appointment AS a
    ON a.doctor_id = d.doctor_id
LEFT JOIN consultation AS c
    ON c.appointment_id = a.appointment_id
LEFT JOIN (
    SELECT
        appointment_id,
        SUM(CASE
                WHEN payment_status = 'Paid' THEN amount
                WHEN payment_status = 'Refunded' THEN -amount
                ELSE 0
            END) AS net_paid
    FROM payment
    GROUP BY appointment_id
) AS pt
    ON pt.appointment_id = a.appointment_id
GROUP BY d.doctor_id, cp.first_name, cp.last_name, s.specialty_name
ORDER BY completed_visits DESC, doctor_name;

-- ------------------------------------------------------------------------
-- TASK 3.5 - UPDATE STATEMENT
-- Purpose: Demonstrate adding an insurance provider to an uninsured patient's
-- profile. ROLLBACK restores the original academic data after verification.
-- ------------------------------------------------------------------------
SET @insurance_before = (
    SELECT COALESCE(insurance_company, 'No insurance')
    FROM patient
    WHERE patient_id = 105
);

START TRANSACTION;

UPDATE patient
SET insurance_company = 'Arabian Shield Cooperative'
WHERE patient_id = 105;

SELECT
    patient_id,
    medical_record_no,
    @insurance_before AS insurance_before_update,
    insurance_company AS insurance_after_update
FROM patient
WHERE patient_id = 105;

ROLLBACK;

SELECT
    patient_id,
    COALESCE(insurance_company, 'No insurance') AS insurance_after_rollback
FROM patient
WHERE patient_id = 105;

-- ------------------------------------------------------------------------
```

7. Appendix B - Complete SQL Implementation

```sql
-- TASK 3.6 - DELETE STATEMENT
-- Purpose: Demonstrate deletion of a pending payment authorization and verify
-- that ROLLBACK restores the row for subsequent project operations.
-- ------------------------------------------------------------------------------
SET @pending_before_delete = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
      AND payment_status = 'Pending'
);

START TRANSACTION;

DELETE FROM payment
WHERE payment_id = 5107
  AND payment_status = 'Pending';

SET @deleted_payment_rows = ROW_COUNT();

SET @pending_after_delete = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
);

ROLLBACK;

SET @pending_after_rollback = (
    SELECT COUNT(*)
    FROM payment
    WHERE payment_id = 5107
);

SELECT
    @pending_before_delete AS rows_before_delete,
    @deleted_payment_rows AS deleted_rows,
    @pending_after_delete AS rows_after_delete,
    @pending_after_rollback AS rows_after_rollback;

-- ------------------------------------------------------------------------------
-- TASK 3.7 - VIEW
-- Purpose: Create a reusable visit-level financial report showing expected
-- charges, net payments, and outstanding balances for every appointment.
-- ------------------------------------------------------------------------------
DROP VIEW IF EXISTS vw_visit_financial_status;

CREATE VIEW vw_visit_financial_status AS
SELECT
    a.appointment_id,
    a.starts_at,
    CONCAT(pp.first_name, ' ', pp.last_name) AS patient_name,
    CONCAT(dp.first_name, ' ', dp.last_name) AS doctor_name,
    s.specialty_name,
    a.appointment_status,
    c.diagnosis,
```

7. Appendix B - Complete SQL Implementation

```
    CASE
        WHEN c.consultation_id IS NULL THEN NULL
        ELSE d.consultation_fee + c.service_fee
    END AS expected_charge,
    COALESCE(SUM(CASE
                    WHEN pay.payment_status = 'Paid' THEN pay.amount
                    WHEN pay.payment_status = 'Refunded' THEN -pay.amount
                    ELSE 0
                END), 0.00) AS net_paid,
    CASE
        WHEN c.consultation_id IS NULL THEN NULL
        ELSE (d.consultation_fee + c.service_fee)
             - COALESCE(SUM(CASE
                                WHEN pay.payment_status = 'Paid' THEN pay.amount
                                WHEN pay.payment_status = 'Refunded' THEN -
pay.amount
                                ELSE 0
                            END), 0.00)
    END AS outstanding_balance
FROM appointment AS a
JOIN patient AS pat
    ON pat.patient_id = a.patient_id
JOIN clinic_person AS pp
    ON pp.person_id = pat.patient_id
JOIN doctor AS d
    ON d.doctor_id = a.doctor_id
JOIN clinic_person AS dp
    ON dp.person_id = d.doctor_id
JOIN specialty AS s
    ON s.specialty_id = d.specialty_id
LEFT JOIN consultation AS c
    ON c.appointment_id = a.appointment_id
LEFT JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
GROUP BY
    a.appointment_id,
    a.starts_at,
    pp.first_name,
    pp.last_name,
    dp.first_name,
    dp.last_name,
    s.specialty_name,
    a.appointment_status,
    c.consultation_id,
    c.diagnosis,
    d.consultation_fee,
    c.service_fee;

SHOW FULL TABLES
WHERE Table_type = 'VIEW';

SELECT *
FROM vw_visit_financial_status
ORDER BY starts_at;
```

7. Appendix B - Complete SQL Implementation

```sql
-- ------------------------------------------------------------------------
-- TASK 3.8 - TRIGGER
-- Purpose: Prevent a prescription from dispensing more units than available
-- and deduct the dispensed quantity from medication inventory automatically.
-- ------------------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_prescription_item_reduce_stock;

DELIMITER $$

CREATE TRIGGER trg_prescription_item_reduce_stock
BEFORE INSERT ON prescription_item
FOR EACH ROW
BEGIN
    DECLARE v_stock_available INT DEFAULT NULL;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_stock_available = NULL;

    SELECT stock_on_hand
    INTO v_stock_available
    FROM medication
    WHERE medication_id = NEW.medication_id;

    IF v_stock_available IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Medication record was not found.';
    ELSEIF NEW.units_dispensed > v_stock_available THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Requested quantity exceeds available clinic
stock.';
    ELSE
        UPDATE medication
        SET stock_on_hand = stock_on_hand - NEW.units_dispensed
        WHERE medication_id = NEW.medication_id;
    END IF;
END$$

DELIMITER ;

SHOW TRIGGERS FROM smart_clinic_system LIKE 'prescription_item';

-- Trigger test: a temporary prescription dispenses four units of medication
-- 3102. The stock changes from 150 to 146, then ROLLBACK restores it to 150.
SET @stock_before_trigger = (
    SELECT stock_on_hand
    FROM medication
    WHERE medication_id = 3102
);

START TRANSACTION;

INSERT INTO prescription
    (prescription_id, consultation_id, issued_at, general_instructions)
VALUES
    (4199, 2104, '2026-07-24 13:45:00',
     'Temporary prescription used to verify the inventory trigger.');
```

7. Appendix B - Complete SQL Implementation

```
INSERT INTO prescription_item
    (prescription_id, medication_id, dose_description, frequency_per_day,
     course_days, units_dispensed, administration_route, item_instructions)
VALUES
    (4199, 3102, 'One capsule', 1, 4, 4, 'Oral',
     'Temporary item used only during the trigger test.');

SET @stock_after_trigger = (
    SELECT stock_on_hand
    FROM medication
    WHERE medication_id = 3102
);

ROLLBACK;

SET @stock_after_rollback = (
    SELECT stock_on_hand
    FROM medication
    WHERE medication_id = 3102
);

SELECT
    @stock_before_trigger AS stock_before_trigger,
    @stock_after_trigger AS stock_after_trigger,
    @stock_after_rollback AS stock_after_rollback;

/*
Optional negative trigger test
------------------------------
Run the following statement separately if evidence of the rejection rule is
required. MySQL should return the custom "Requested quantity exceeds available
clinic stock" error. It is commented out so the complete script runs without an
intentional failure.

INSERT INTO prescription_item
    (prescription_id, medication_id, dose_description, frequency_per_day,
     course_days, units_dispensed, administration_route, item_instructions)
VALUES
    (4101, 3102, 'Invalid test quantity', 1, 1, 9999, 'Oral',
     'This row must be rejected by the trigger.');
*/
-- ============================================================================
-- END OF SMART CLINIC DATABASE IMPLEMENTATION
-- ============================================================================
```

8. Appendix C - Screenshot Evidence Index

This index maps every embedded MySQL Workbench screenshot to the Task 2 or Task 3 requirement it supports.

No.	Evidence Purpose	Source File
2.1	Database object inventory: ten base tables and one view	01_database_objects.jpg
2.2	CLINIC_PERSON populated table (10 rows)	02_clinic_person_table.jpg
2.3	PATIENT populated table (5 rows)	03_patient_table.jpg
2.4	SPECIALTY populated table (5 rows)	04_specialty_table.jpg
2.5	DOCTOR populated table (5 rows)	05_doctor_table.jpg
2.6	APPOINTMENT populated table (7 rows)	06_appointment_table.jpg
2.7	CONSULTATION populated table (5 rows)	07_consultation_table.jpg
2.8	MEDICATION populated table (7 rows)	08_medication_table.jpg
2.9	PRESCRIPTION populated table (5 rows)	09_prescription_table.jpg
2.10	PRESCRIPTION_ITEM populated table (7 rows)	10_prescription_item_table.jpg
2.11	PAYMENT populated table (7 rows)	11_payment_table.jpg
3.1	Completed appointments SELECT result (5 rows)	12_select_completed.jpg
3.2	Low-stock medications SELECT result (4 rows)	13_select_low_stock.jpg
3.3	Appointment details JOIN result (7 rows)	14_join_appointments.jpg
3.4	Patient prescriptions JOIN result (7 rows)	15_join_prescriptions.jpg
3.5	Nested query: patients above average paid total (IDs 102 and 105)	16_nested_payments.jpg
3.6A	Doctor aggregate and GROUP BY evidence - Part A	17_group_by_part_a.jpg
3.6B	Doctor aggregate and GROUP BY evidence - Part B	18_group_by_part_b.jpg
3.7	UPDATE and ROLLBACK verification	19_update_rollback.jpg
3.8	DELETE and ROLLBACK verification	20_delete_rollback.jpg
3.9	Financial VIEW result (7 rows)	21_financial_view.jpg
3.10A	TRIGGER stock-control test - Part A	22_trigger_part_a.jpg
3.10B	TRIGGER stock-control test - Part B and final result	23_trigger_part_b.jpg